ARCHIMESH

MANUAL

October 28, 2026

Contents

List of Figures **6**

1 Introduction **7**

1.1 What Archimesh offers 7

1.2 What this manual covers 7

1.3 Terminology 8

2 Getting Started **9**

2.1 Prerequisites 9

2.2 Configuration 9

2.3 Starting infrastructure 9

2.4 Building and running the server 10

2.5 Creating a model 10

2.6 Building the client plugin 10

2.7 Building the admin UI 10

2.8 First collaborative session 11

2.9 Validation scripts 11

2.10 Stopping 11

3 Architecture **12**

3.1 System purpose 12

3.2 High-level flow 12

3.3 Component layout 12

3.4 Server component map 13

3.5 Client plugin component map 13

3.6 Two kinds of state 14

3.7 Snapshot vs op-log 14

3.8 Folder model 14

3.9 Linear timeline 14

3.10 Deployment and operational considerations 16

4 Client Plugin 18

4.1 Plugin entry point 18

4.2 Session manager 18

4.3 EMF change capture 19

4.4 Operation mapping 19

4.5 Remote operation application 20

4.6 Offline outbox 20

4.7 Cache policy 22

4.8 Accepted-echo reconciliation 22

4.9 Menu commands 22

5 Server API 23

5.1 WebSocket endpoint 23

5.2 REST API 25

6 Persistence 28

6.1 Repository layer 28

6.2 Graph model 28

6.3 Neo4j constraints 31

6.4 Persistence invariants 32

6.5 Configuration 32

7 Messaging 33

7.1 Topic layout 33

7.2 Keyed ordering and capacity 33

7.3 Producer 33

7.4 Consumer 34

7.5 Retention 34

7.6 Publication failure and duplicate delivery 34

7.7 Configuration 36

8 CRDT and Merge Semantics 37

8.1 Causal identity 37

8.2 Merge types by domain 37

8.3 Delete semantics — tombstones 38

8.4 Notation field validation . . . 38
8.5 Lock model . . . 39
8.6 CRDT maturity . . . 39
9 Authorization . . . 40
9.1 Identity modes . . . 40
9.2 Role model . . . 40
9.3 Policy model . . . 41
9.4 JWT and OIDC setup . . . 41
9.5 Authorization diagnostics . . . 42
9.6 Configuration summary . . . 42
10 Administration and Supervision . . . 43
10.1 Admin UI . . . 43
10.2 Audit logging . . . 43
10.3 Supervision checklist . . . 44
11 System Management . . . 45
11.1 Model lifecycle . . . 45
11.2 Recovery checkpoints . . . 45
11.3 Compaction and payload pruning . . . 46
11.4 Optional scheduled maintenance . . . 47
11.5 Export . . . 49
11.6 Import . . . 51
11.7 Rebuild . . . 52
11.8 Tags . . . 52
11.9 Integrity checks . . . 53
11.10 Consistency checks . . . 53
11.11 ACL management . . . 53
11.12 Status monitoring . . . 54
12 Troubleshooting . . . 55
12.1 Diagnostic endpoints . . . 55
12.2 Common problems . . . 55
12.3 Log analysis . . . 57

12.4 Recovery procedures 58
12.5 Performance tuning 59
13 Script Reference 60
13.1 Build and install 60
13.2 Development infrastructure 60
13.3 Validation and gate checks 60
13.4 Log monitoring 61
14 Wire Protocol and Schema Reference 62
14.1 Schema files 62
14.2 Message flow summary 62
14.3 Common type definitions 63
14.4 Op batch envelope 63
14.5 Notation JSON objects 63
14.6 Error codes 64
14.7 Export/import format 64

List of Figures

Figure 1: System components and transport boundaries.. 15

Figure 2: Checkout and reconnect decisions, including snapshot fallback after pruning. 21

Figure 3: Durable acceptance and asynchronous operation fan-out. 35

Figure 4: Recovery from a checkpoint and complete tail; receipts preserve retry identity. 46

Figure 5: Payload pruning gates, publication fencing and retained safety state. 48

Figure 6: Optional per-model maintenance phases and policy gates. 50

1 Introduction

Archimesh is a collaborative real-time modelling server for Archi, the ArchiMate modelling tool. It lets multiple modellers work on the same model simultaneously, with the server acting as the authoritative source of truth for the shared model timeline.

This manual is written for developers and operators who need to set up, run, supervise and troubleshoot an Archimesh deployment. It assumes familiarity with Java, Docker, Neo4j and Kafka at a working level.

1.1 What Archimesh offers

Archimesh provides five core capabilities:

- **Real-time collaborative editing.** Multiple Archi clients connect to one server, capture local edits as structured operations, and apply remote operations as they are committed. The server assigns a single authoritative revision order so every client converges on the same model state.
- **Authoritative model storage.** Neo4j stores both the current materialized graph, accepted history, recovery checkpoints and merge metadata. The server is the source of truth; local Archi files are treated as cache projections.
- **CRDT-informed merge.** The server applies Last-Writer-Wins (LWW) per-field merges with causal clocks and tombstone protection for concurrent edits on the same diagram objects. Lease-based locks protect noisy notation edits (resize, move, bendpoints, style) from destructive interleaving.
- **Offline resilience.** When a client loses its connection, local edits are queued in a bounded persistent outbox and replayed in FIFO order after reconnect. Replayed ops are rebased to the latest server revision before send. Transport retries preserve pending intent; correlated semantic/lock conflicts are surfaced to the user and trigger projection recovery.
- **Operational visibility.** The admin API and admin UI expose model catalog, active sessions, recent activity, ACL management, audit trails, integrity checks, compaction control, and export/import for backup and migration.

1.2 What this manual covers

- Section 2 walks through building and running the server and client locally.
- Section 3 describes the system architecture, component layout and major data flows.
- Section 4 covers the Archi client plugin: EMF change capture, operation mapping, remote apply, offline outbox and cache policy.
- Section 5 documents the WebSocket and REST API endpoints, message types and request/response contracts.
- Section 6 explains the Neo4j graph model: materialized state, op-log, tags, tombstones and property clocks.
- Section 7 covers the Kafka topic layout, partitioning requirements and fan-out path.
- Section 8 details the CRDT merge semantics: LWW field merges, tombstone protection, OR-set collections and lock model.
- Section 9 documents identity modes, role normalization, per-model ACLs and the policy decision point.
- Section 10 explains the admin UI and operational supervision: overview, models, versions, sessions, access and audit.

- Section 11 covers system management: creating models, compactions, exports, imports, rebuilds, tags and integrity checks.
- Section 12 describes common failure modes, log analysis, diagnostic endpoints and recovery procedures.
- Appendix 13 is a reference for all scripts in the `scripts/` directory.
- Appendix 14 documents the wire protocol message types and JSON schema contracts.

1.3 Terminology

Model A named linear revision timeline. One `HEAD`, zero or more immutable tags, no branches. Identified by a stable `modelId`.

Operation (op) A single Archimesh change: create, update or delete of an element, relationship, view, view object, connection, folder or property.

Op batch A client-submitted group of operations sharing one `opBatchId` for idempotency. The server assigns one contiguous revision range to the batch.

Revision A monotonically increasing timeline position. Each op in a batch consumes one revision. The revision range is the authoritative ordering.

HEAD The current tip of the model timeline. Only `HEAD` is writable; tagged revisions are read-only.

Tag An immutable named pointer to a historical revision, stored with a snapshot of the model state at that point.

Materialized state The current model graph in Neo4j, kept in sync with the op-log. Used for fast joins, exports and admin inspection.

Outbox A bounded persistent FIFO queue on the client side that holds local edits while disconnected from the server.

Checkpoint An immutable, checksummed raw projection at committed revision C, including clocks, memberships and tombstones. Recovery restores this base and replays the complete operation tail C+1 through HEAD.

Acceptance receipt Retained batch ID and original revision range, preserving retry deduplication after its operation payload has been pruned.

Compaction Admin-controlled pruning of checkpoint-covered, published operation payloads below a retention watermark. Acceptance receipts and all merge metadata are retained for recovery and stale-write safety.

2 Getting Started

This section walks through building and running Archimesh locally. It assumes Docker Desktop (or Docker Engine with the Compose plugin) and Java 21+.

2.1 Prerequisites

- Docker Desktop (or Docker Engine + Compose plugin)
- Java 21+ (for the Quarkus server)
- Maven for the server; the client uses its pinned Maven 3.9.16 wrapper
- Eclipse with Archi and the Archi modelling tool (for the client plugin)

2.2 Configuration

1. Copy `.env.example` to `.env`:

```
cp .env.example .env
```

Listing 1: Setup environment

2. Edit `.env` if you need to change the Neo4j password or Kafka bootstrap address. The defaults work for local development:

```
NEO4J_USER=neo4j
NEO4J_PASSWORD=devpassword
NEO4J_URI=bolt://localhost:7687
KAFKA_BOOTSTRAP_SERVERS=localhost:9092
KAFKA_TOPIC_PREFIX=archimesh.model
KAFKA_CONSUMER_GROUP_ID=archimesh-server
ARCHIMESH_CONSISTENCY_CHECK=false
ARCHIMESH_AUTHZ_ENABLED=false
ARCHIMESH_IDENTITY_MODE=bootstrap
ARCHIMESH_LOG_LEVEL=INFO
```

Listing 2: Default .env values

2.3 Starting infrastructure

Start Neo4j and Kafka with Docker Compose:

```
docker compose up -d
```

Listing 3: Start containers

Services available after startup:

Neo4j Browser `http://localhost:7474`
Neo4j Bolt `bolt://localhost:7687`
Kafka broker `localhost:9092`
Kafka UI `http://localhost:8080`

Apply Neo4j constraints and indexes:

```
docker compose exec -T neo4j cypher-shell \
  -u "${NEO4J_USER:-neo4j}" \
  -p "${NEO4J_PASSWORD:-devpassword}" \
  < neo4j/schema.cypher
```

Listing 4: Apply schema

2.4 Building and running the server

The server is a Quarkus application in `server/`:

```
cd server
mvn quarkus:dev
```

Listing 5: Build and run server

The server starts on port 8081. Verify:

```
curl -s http://localhost:8081/admin/overview | jq .
```

Listing 6: Health check

2.5 Creating a model

Models must be explicitly registered before use:

```
curl -s -X POST http://localhost:8081/admin/models/demo | jq .
```

Listing 7: Register model

2.6 Building the client plugin

The client plugin is an Eclipse RCP bundle:

```
cd client
./mvnw -q -pl org.gautelis.archimesh.plugin -am install -DskipTests
```

Listing 8: Build client plugin

Install the built plugin into Archi using:

```
./scripts/install-archimesh-plugin.sh
```

Listing 9: Install plugin

2.7 Building the admin UI

The admin UI is a Svelte application:

```
cd admin-web
npm install
npm run build
```

Listing 10: Build admin UI

Or use the convenience script:

```
./scripts/build-admin-web.sh
```

Listing 11: Build admin UI via script

The built static app is served by Quarkus at `/admin-ui/`.

2.8 First collaborative session

1. Start two Archi instances with the plugin installed.
2. In the first instance, create a new model or open an existing one.
3. Use **Tools > Connect Archimesh...** to connect. Enter the WebSocket URL (e.g. `ws://localhost:8081`) and model id (`demo`).
4. In the second instance, connect to the same model.
5. Make edits in either instance. Changes appear in the other within the broadcast latency of the Kafka fan-out path (typically under one second).

2.9 Validation scripts

The project includes local validation scripts for fast feedback:

```
# Fast hardening and parity checks (no Docker infra needed)
./scripts/validate-local.sh fast

# Full client + server gate
./scripts/validate-local.sh gate

# Full gate with local Kafka/Neo4j-backed integration tests
./scripts/validate-local.sh infra

# Full gate with websocket end-to-end coverage
./scripts/validate-local.sh ws

# Isolated packaged JVM crash and fresh-database restore drill
./scripts/validate-local.sh recovery
```

Listing 12: Validation commands

These are the supported release gate entrypoints. Run `fast` before committing; run `gate` or `infra` before pushing.

2.10 Stopping

```
docker compose down
```

Listing 13: Stop infrastructure

Add `-v` to also remove volumes (clean slate):

```
docker compose down -v
```

Listing 14: Stop and remove volumes

Additional local setup notes are in `dev-setup.md`. The introduction maps the remaining chapters to developer and operator tasks.

3 Architecture

This chapter describes the overall system architecture, component layout and major data flows in Archimesh.

3.1 System purpose

The system lets multiple Archi clients collaborate on the same model timeline through a centralized server. It combines:

- a local Archi client plugin that captures and applies model changes
- a Quarkus server that validates, orders, persists and broadcasts changes
- Kafka for ordered model event fan-out
- Neo4j for the materialized model graph, op-log, tags and merge metadata

The model timeline is intentionally linear: one `HEAD`, no branches, immutable tags as named historical points.

Figure 1 shows the client and server boundaries. Only Quarkus connects to Neo4j and Kafka; Archi plugins and the browser use the server's WebSocket and HTTP interfaces.

3.2 High-level flow

1. A user edits a model in Archi.
2. The client plugin captures local EMF changes and converts them into Archimesh operations.
3. The server validates the op batch against preconditions, locks and idempotency state, then assigns authoritative revisions.
4. One Neo4j transaction commits the log, projection, head revision and durable publication intent. The submitting session receives acceptance.
5. The delivery worker publishes committed operations to Kafka in model revision order, retrying failures from durable state.
6. Other connected clients apply the accepted operations into their local Archi model.

3.3 Component layout

Component	Responsibility
`server/`	Quarkus server: WebSocket and REST endpoints, authorization, validation, revision assignment, idempotency, persistence, broadcast.
`client / org . gautelis . archimesh . plugin/`	Eclipse RCP plugin: WebSocket client, EMF change capture, operation mapping, remote apply, offline outbox, cache persistence.
`client / client - tests/`	Client plugin tests: CRDT merge, operation mapping, folder sync, outbox replay, notation inventory.
`admin-web/`	Svelte admin UI: model overview, catalog, versions, access control, sessions, audit.

schemas/	JSON schemas for common types, op batches, operations, lock events, presence and notation contracts.
neo4j/	Cypher schema for constraints and indexes, plus a documented graph model.
scripts/	Build, validation, development and operational scripts.
docker-compose.yml	Local dev infrastructure: Neo4j + Kafka.

3.4 Server component map

The server is organized into distinct packages:

Package	Responsibility
endpoint	WebSocket endpoint (ArchimeshEndpoint), REST endpoints (ModelStateEndpoint, AdminEndpoint, AdminUiEndpoint, ConsistencyEndpoint).
service	Central coordination (ArchimeshService), plus service interfaces for validation, revision, locks, Neo4j repository, Kafka publish/consume, session registry and idempotency.
CommitDeliveryService	Writer-lease renewal and ordered retry of committed publication intents.
CheckpointMaintenanceService	Optional scheduled checkpoint creation, covered payload pruning and older-checkpoint retention.
service.impl	Concrete implementations: in-memory services for validation, revision, locks and sessions; Neo4j repository with split support classes for op-log, materialized state, reads, checkpoints and compaction; Kafka publisher and consumer implementations.
auth	Authorization: identity modes, role normalization, policy decision point (PDP), authorization service and transport.
model	Admin data classes: status, catalog, sessions, ACLs, export/import, compaction, integrity, audit.
wire	Wire protocol envelopes (ClientEnvelope, ServerEnvelope) and message types for inbound (Join, SubmitOps, AcquireLock, ReleaseLock, Presence) and outbound (CheckoutSnapshot, CheckoutDelta, OpsAccepted, OpsBroadcast, LockEvent, PresenceBroadcast, Error).

3.5 Client plugin component map

Package	Responsibility
ws	WebSocket session manager, outbox handling, join/rejoin decisions, local submit tracking, inbound message dispatch.
emf	EMF change capture, operation mapping, remote op application, CRDT merge (LWW, OR-set, tombstones), model controller.
startup	Eclipse startup hooks, workbench lifecycle bridge, status line bridge.
handlers	Menu command handlers: connect, disconnect, open server model, switch active model, show last conflict, resynchronize.
ui	Connect dialog, server model catalog dialog, open server model dialog.
notation	Notation serializer and deserializer for Archi diagram fields.

`util`	Authentication hint helpers and lightweight JSON utilities.

3.6 Two kinds of state

The system maintains two kinds of state at all times:

Local working state Inside each Archi client. This is the EMF model the user sees and edits. It is not authoritative.
Authoritative shared state On the Archimesh server. The Neo4j graph is the source of truth. A verified checkpoint and complete operation tail, or the full log before the first checkpoint, provide recovery data.

The client plugin synchronizes between these two states: capturing local changes, submitting them to the server, and applying remote changes into the local model.

3.7 Snapshot vs op-log

The server keeps a materialized graph, accepted operation history and optional recovery checkpoints in Neo4j. Their roles differ:

- The materialized graph makes joins, exports and admin inspection fast (no replay needed).
- Retained batch payloads provide delta checkout and replay after a checkpoint.
- A raw checkpoint preserves projection state and all merge metadata at a committed revision. Earlier covered payloads can then be pruned.
- Compact acceptance receipts preserve batch identity and revision ranges after payload pruning. Tags retain independent historical snapshots.

Exports contain the current client-facing snapshot and either complete history (v1), or a checkpoint, prefix receipts and complete tail (v2). Checkpoints contain raw graph and merge state beyond the client snapshot. Arbitrary historical replay before the recovery base requires an archived full history; tagged snapshots remain available independently.

3.8 Folder model

Model-tree folders are first-class synchronized state with strict rules:

- Root folders have stable synthetic ids (e.g. `folder:root-business`).
- User folders have immutable ids — rename is allowed, but identity is stable.
- Object placement in folders is explicit and persisted.
- Cross-root folder moves are rejected by validation.
- Non-empty folder delete is rejected.

This avoids drift between clients and preserves Archi model-tree structure.

3.9 Linear timeline

Every model has exactly one linear revision timeline with:

- One moving `HEAD`

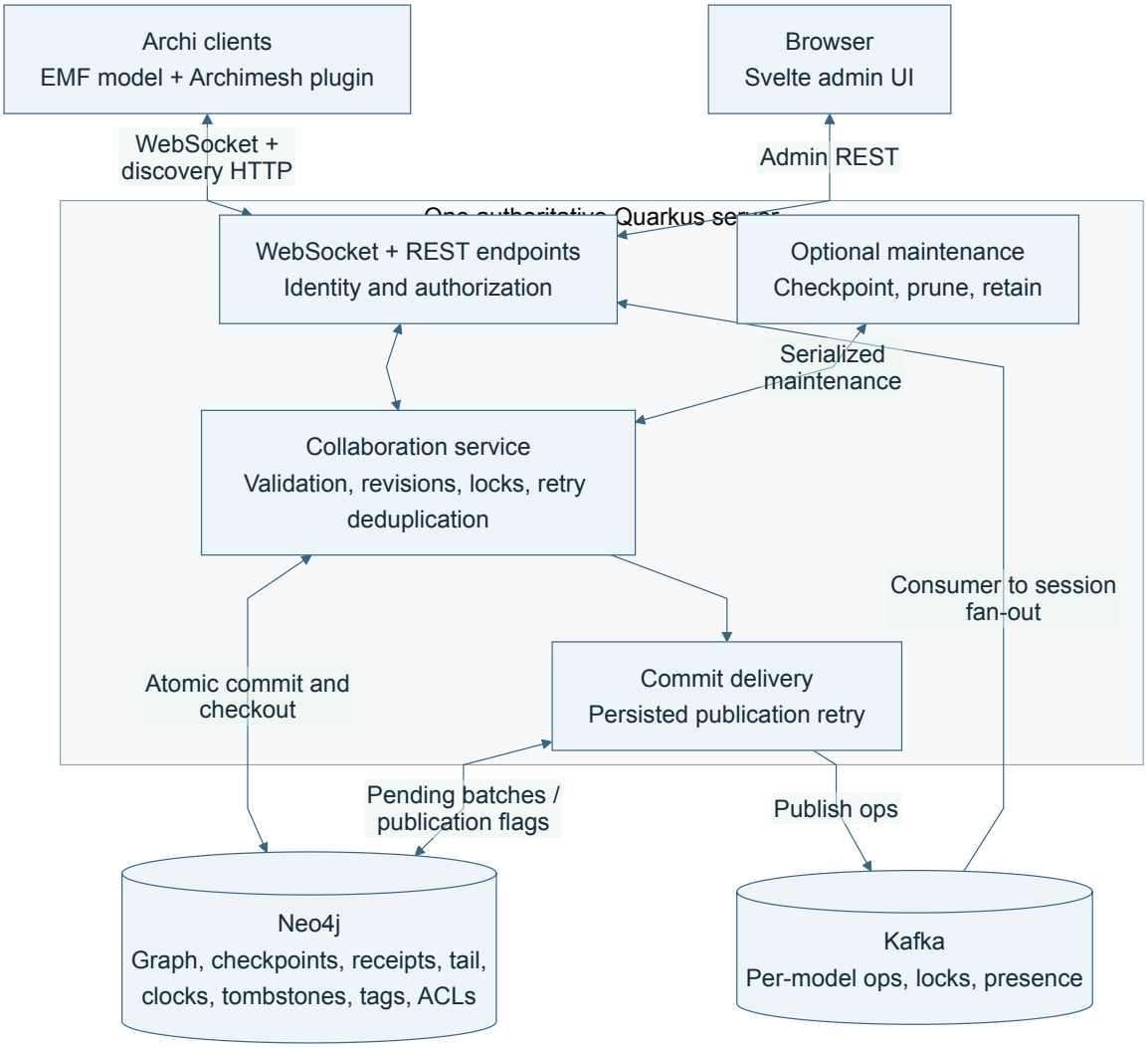


Figure 1: System components and transport boundaries.

- Zero or more immutable tags
- No branches

Clients submit batches against a `baseRevision`. The server assigns an authoritative contiguous revision range. If a batch contains N ops it consumes N revisions. Tags are read-only pointers to historical revisions, stored with a snapshot of the model at that point.

Only `HEAD` is writable. Tagged revisions can be read but reject writes.

3.10 Deployment and operational considerations

3.10.1 Authoritative writer and durability

Run one authoritative server per Neo4j repository. A renewable database lease rejects a second active writer; commit and maintenance transactions check the lease, and timeline operations are serialized within the active service. Active-active server operation is not supported. Locks, presence and live session registries are process-local; committed state and publication intents survive restart in Neo4j. See Section 7 for delivery and Section 11 for checkpoint recovery.

3.10.2 Transport security

The WebSocket and REST endpoints do not terminate TLS themselves. In production, deploy behind a reverse proxy (Nginx, Caddy, Traefik) that terminates TLS and forwards to the Quarkus server. Example configurations are provided in `server/examples/`.

When using `app.identity.mode=proxy`, the reverse proxy must also forward trusted identity headers, and the WebSocket upgrade path must carry those headers through the proxy to the server.

3.10.3 Reconnection backpressure

After a server restart, all previously connected clients may attempt to reconnect simultaneously. To avoid overwhelming the server:

- Clients should apply jitter to reconnection attempts (randomized delay 1–5 seconds before the first retry).
- The server should reject joins that arrive faster than a configurable rate limit.
- Admin diagnostics expose active session counts so operators can detect reconnection storms.

The current client implementation does not yet include reconnect jitter. This is a known operational gap for production deployments with many concurrent clients.

3.10.4 Kafka keyed ordering

Three shared topics carry operations, locks and presence. Every record is keyed by model id, keeping each model in one partition per topic while partition counts remain stable. One partition per topic is sufficient for local development, giving three application partitions regardless of model count. Larger fixed counts distribute throughput; do not resize during live publication without draining and coordinating a cutover. Models share partition capacity, and events in different topics have no common order. The old per-model topic layout is no longer consumed.

If Kafka publication fails, the committed payload remains pending in Neo4j and is retried by the delivery worker. The submitting client may already have received acceptance while other clients await fan-out. There is no direct broadcast bypass in the normal production delivery path.

4 Client Plugin

The client plugin is an Eclipse RCP bundle that runs inside Archi. It captures local EMF model changes, submits them to the Archimesh server, and applies remote operations into the local model.

4.1 Plugin entry point

The plugin activator is `org.gautelis.archimesh.plugin.ArchimeshPlugin`, an `AbstractUIPlugin` subclass.

On startup it creates the three central objects:

`ArchimeshSessionManager` WebSocket lifecycle, outbox handling, join/rejoin decisions and local submit tracking.
`WorkbenchModelLifecycleBridge` Hooks into Archi's workbench to detect model open, close, save and dirty-state changes.
`ArchimeshStatusLineBridge` Displays connection state, conflict banners and outbox capacity warnings in the Archi status line.

The plugin id is `org.gautelis.archimesh.plugin`. Debug and trace logging are controlled by the system property `archimesh.debug` and `archimesh.trace`, or equivalently the environment variables `ARCHIMESH_DEBUG` and `ARCHIMESH_TRACE`.

4.2 Session manager

`ArchimeshSessionManager` is the central client-side coordinator. It manages:

- WebSocket connection lifecycle (connect, disconnect, reconnect)
- Per-model persistent bounded outbox (FIFO queue with replay)
- Join/rejoin decision logic with cache metadata
- Accepted-echo reconciliation for locally submitted op batches
- Cache autosave policy (debounced saves on sync traffic)

4.2.1 Connection flow

1. The user selects **Tools > Connect Archimesh...** and provides the server URL, model id and credentials.
2. The session manager resolves the join decision:
   - If the session is server-backed, cache metadata is read from `~/Archi/archimesh-cache/<modelId>.meta.properties`.
   - If metadata is valid and the cached revision is known, the join includes `lastSeenRevision` so the server can send a delta.
   - If metadata is missing, mismatched or the revision is unknown, the join omits `lastSeenRevision` to request a cold snapshot.
3. The session manager opens a WebSocket to `<baseUrl>/models/<modelId>/stream` and sends a `Join` message.
4. The server responds with `CheckoutSnapshot` (full model state) or `CheckoutDelta` (incremental changes since lastSeenRevision).

5. The client applies the checkout payload into the local Archi model.
6. Only after checkout completes does it drain pending submissions.

The server checks delta coverage, not just the latest available revision. Missing history after pruning or an ahead-of-server revision causes a current snapshot. Snapshots restore values, clocks and tombstones before queued work is replayed. Figure 2 shows the decision path.

4.2.2 Rejoin and cache verification

On reconnect the session manager reads the cache metadata file. A cache projection is discarded (forcing a cold snapshot) if:

- The metadata file does not exist or cannot be read.
- The metadata `serverBacked` flag does not match.
- The `modelId` or `modelRef` does not match the target.
- The cache file is missing from disk.
- The cached revision is unknown (negative).
- The cache file exists but is implausibly small for a model with revision ≥ 1 (under 1 KB — likely truncated or corrupted).

After join, the server's first revision hint is compared against the cache revision used at join. If the server revision is behind the cache revision, the client requests a cold snapshot rebuild.

4.3 EMF change capture

`EmfChangeCapture` observes local EMF model changes via Archi's ECore notification mechanism. When the user edits the model, the capture layer:

1. Receives EMF notifications for element, relationship, view, folder and property changes.
2. Batches correlated changes (e.g. a view object create + its initial geometry update) into a single logical operation.
3. Defers connection-creation submission until all referenced identifiers (source, target, relationship, view) are available, then emits the relationship and connection in a single `SubmitOps` batch.
4. Passes the collected operations to `OpMapper` for wire-format encoding.

The capture layer runs under a remote-apply guard so that operations applied from the server are not re-captured and re-submitted as local edits.

4.4 Operation mapping

`OpMapper` converts captured EMF changes into Archimesh wire-format operations. The supported operation types are:

Operation	Purpose
`CreateElement`	Create a new ArchiMate element.
`UpdateElement`	Update element name or documentation.
`DeleteElement`	Delete an element and cascade-delete its diagram representations.

CreateRelationship	Create a relationship between two elements.
UpdateRelationship	Update relationship fields.
DeleteRelationship	Delete a relationship and cascade-delete its connections.
CreateFolder	Create a user folder.
UpdateFolder	Rename a folder or update its documentation.
DeleteFolder	Delete an empty user folder.
CreateView	Create a new diagram view.
UpdateView	Update view metadata.
DeleteView	Delete a view and its contents.
CreateViewObject	Place an element on a diagram.
UpdateViewObjectOpaque	Update geometry, style or label fields of a diagram object.
DeleteViewObject	Remove a diagram object.
CreateConnection	Draw a connection between two diagram objects.
UpdateConnectionOpaque	Update connection style, label or bendpoint fields.
DeleteConnection	Remove a diagram connection.
MoveElementToFolder	Move an element to its destination folder.
MoveRelationshipToFolder	Move a relationship to its destination folder.
MoveViewToFolder	Move a view to its destination folder.
SetProperty	Set a key-value property on a model entity.
UnsetProperty	Remove a property.
AddPropertySetMember	Add an element to a property set.
RemovePropertySetMember	Remove an element from a property set.
AddViewObjectChildMember	Nest a view object inside another.
RemoveViewObjectChildMember	Remove nested view object membership.

Each operation carries a `causal` object with `clientId`, `lamport` and `opId` for LWW merge semantics.

4.5 Remote operation application

`RemoteOpApplier` applies snapshots and remote broadcasts into the local EMF model. Key behaviors:

- Applies operations under a remote-apply guard to prevent re-capture.
- Applies per-field LWW merges for notation fields (geometry, style, labels) using causal clock comparison.
- Maintains element field clocks and tombstones for delete safety.
- Handles property-level LWW and OR-set semantics for `SetProperty`, `UnsetProperty`, `AddPropertySetMember`, `RemovePropertySetMember`.
- Restores folder structure before folder memberships during snapshot application.
- Restores views before view objects and connections.
- Defers dependency-sensitive operations when referenced entities are not yet created.

4.6 Offline outbox

Every submission is persisted in the bounded outbox before a send, including while online. Key guarantees:

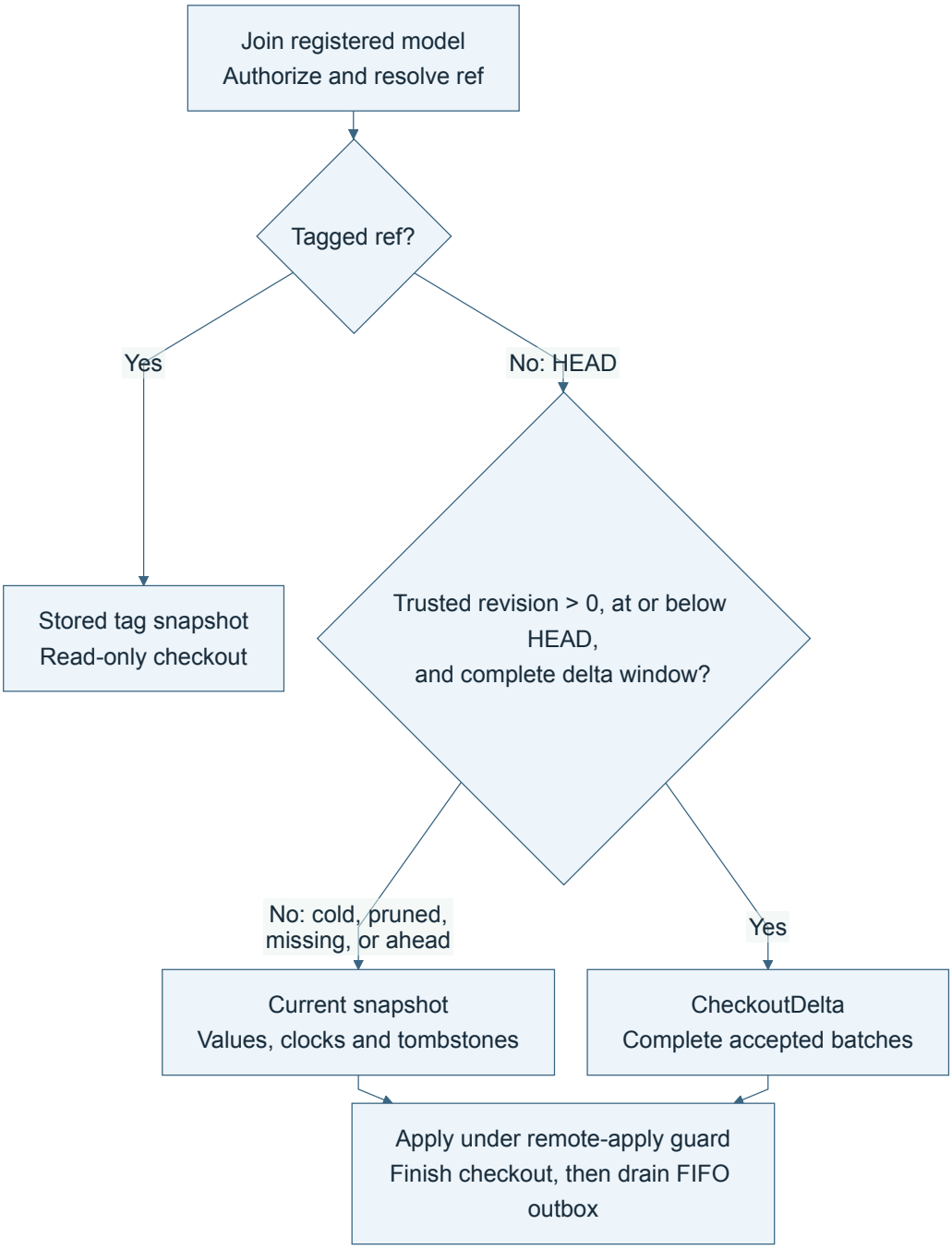


Figure 2: Checkout and reconnect decisions, including snapshot fallback after pruning.

- **FIFO per model.** Operations are drained in submission order.
- **Rebase before send.** Each replayed op has its `baseRevision` updated to the current known server revision.
- **Ack-gated removal.** Entries are removed only after the server sends a matching `OpsAccepted`. Websocket send completion is not enough — the server must confirm commit.
- **Transport retry.** Failed sends and missing acknowledgments retry with capped exponential backoff. Transport failures preserve the entry and its batch ID so the server can return an earlier acceptance.
- **Conflict resolution.** Entries receiving `PRECONDITION_FAILED` or `LOCK_CONFLICT` from the server remove the correlated conflicting entry, surface a conflict callback in the status line, and trigger projection recovery. Another client's rejection does not remove local work.
- **Bounded queue.** Maximum 1000 entries. At capacity, new enqueue is rejected while existing uncommitted work is preserved. A warning fires at 80% capacity (800 entries) and is displayed in the status line.
- **Durable persistence.** The outbox is persisted to `~/Archi/archimesh-cache/<modelId>.archimate.outbox.properties` and reloaded on reconnect.

4.7 Cache policy

When running in server-backed mode, the client treats the local `.archimate` file as a cache projection, not the primary artifact.

Cache path `~/Archi/archimesh-cache/<modelId>.archimate`
Metadata file `<cacheFile>.meta.properties` stores `modelId`, `modelRef`, `lastKnownRevision`, `serverBacked` and `savedAtEpochMs`.
Autosave triggers Debounced saves (2 s cooldown) after `CheckoutSnapshot`, `CheckoutDelta`, `OpsBroadcast` and `OpsAccepted`.
Force-save triggers On model attach, detach and disconnect.
Discard policy Cache is discarded if inconsistent, invalid or not server-backed at reconnect.

4.8 Accepted-echo reconciliation

Local edits are optimistic. The client seeds local merge clocks, advances its logical clock from incoming causal metadata, and reconciles its own accepted broadcast against the current local state. It does not simply discard every self-echo: accepted server normalization and merge metadata must be reflected without overwriting a newer optimistic edit. Applied revision advances after application. Acceptance removes the matching outbox entry; it can arrive before or after the corresponding broadcast.

4.9 Menu commands

The plugin contributes the following commands to Archi's **Tools** menu:

Connect Archimesh... Enter server URL, model ID, identity and optional bearer token.
Disconnect Archimesh Save cache and return to local mode.
Open Archimesh Model from Server... Browse and open a readable server model.
Switch Active Model to Server-Backed... Attach the active local model.
Show Last Archimesh Conflict Inspect the latest conflict.
Resynchronize Model Replace the projection from a fresh server snapshot.

5 Server API

This chapter documents the WebSocket and REST API endpoints exposed by the Archimesh server.

5.1 WebSocket endpoint

Path `/models/\{modelId\}/stream`
Protocol WebSocket (RFC 6455)
Sub-protocol None (plain JSON text frames)

The WebSocket endpoint handles real-time collaboration. Each connection is scoped to one model.

5.1.1 WebSocket lifecycle

1. Client opens a WebSocket connection to the endpoint.
2. Server performs authorization checks (see Section 9) and either accepts the connection or closes it with policy violation status.
3. Client sends a `Join` message to request model state.
4. Server responds with `CheckoutSnapshot` or `CheckoutDelta`.
5. Client sends `SubmitOps`, `AcquireLock`, `ReleaseLock` and `Presence` messages during the session.
6. Server replies to the submitting session with `OpsAccepted` or a correlated `Error`. Model subscribers receive `OpsBroadcast`, `LockEvent` and `PresenceBroadcast`.
7. Client or server closes the WebSocket. On close the server unregisters the session and releases held locks.

5.1.2 Inbound messages (client to server)

All inbound messages are wrapped in a `ClientEnvelope`:

```
{
  "type": "<MessageType>",
  "payload": { ... }
}
```

Listing 15: ClientEnvelope shape

Type	Purpose	Key payload fields
`Join`	Request model checkout	`lastSeenRevision` (optional), `ref` (optional; `HEAD` or tag name), `actor` (`userId`, `sessionId`)
`SubmitOps`	Submit one batch of operations	`baseRevision`, `opBatchId`, `actor`, `ops` (array)
`AcquireLock`	Acquire a notation lease	`actor`, `targets` (array of entity ids), `ttlMs`
`ReleaseLock`	Release a notation lease	`actor`, `targets` (array of entity ids)

Presence	Broadcast cursor/selection	actor, viewId, selection (array), cursor (object)

5.1.3 Join and checkout

When the server receives a Join:

1. The model must exist (registered via admin API).
2. Authorization is checked (join action).
3. The requested ref is resolved (HEAD or tag name).
4. Tagged refs always receive their stored read-only snapshot. For HEAD, a positive trusted client revision at or below head can receive a delta only when every range and operation count in the requested window is present. A gap after pruning falls back to a current snapshot.
5. Otherwise, the server responds with CheckoutSnapshot containing the full materialized model state at the requested ref.

5.1.4 SubmitOps processing

The SubmitOps pipeline (implemented in ArchimeshService):

1. Model registration and writable ref checks.
2. Validation of the op batch structure.
3. Normalization and expansion (bootstrap view ref normalization, cascade delete expansion).
4. Durable idempotency check: an existing batch receipt returns its original OpsAccepted range, including after payload pruning.
5. Revision-ahead check: if baseRevision > headRevision, send Error(REVISION_AHEAD) to the submitting session.
6. Precondition validation: all entity references must exist either in persisted state or in earlier ops within the same batch.
7. Lock conflict check: for operations requiring notation locks (UpdateViewObjectOpaque, UpdateConnectionOpaque), verify no other session holds a conflicting lock.
8. Revision range assignment: allocate from..to consuming one revision per op.
9. Causal metadata normalization: stamp each op with clientId, lamport, opId.
10. Atomic persistence: append log, apply projection, update head and store pending publication payload in one Neo4j transaction. Failed persistence receives no acceptance and invalidates the in-memory revision cache.
11. Cache the committed acceptance range.
12. Optional post-commit consistency checks.
13. Request the durable delivery worker to publish committed batches.
14. Reply OpsAccepted to the submitting session.

If validation rejects the batch, a correlated Error is sent to its submitter with a specific code (REVISION_AHEAD, PRECONDITION_FAILED, LOCK_CONFLICT).

5.1.5 Outbound messages (server to client)

Type	Purpose	Key payload fields
`CheckoutSnapshot`	Full model state	`modelId`, `headRevision`, `elements`, `relationships`, `views`, `folders`, etc.
`CheckoutDelta`	Incremental changes	`modelId`, `fromRevision`, `toRevision`, `ops` (array)
`OpsAccepted`	Batch commit confirmation	`opBatchId`, `baseRevision`, `assignedRevisionRange`
`OpsBroadcast`	Remote batch broadcast	`opBatch` (full committed batch)
`LockEvent`	Lock state change	`action` (acquired/released/expired), `actor`, `targets`
`PresenceBroadcast`	Remote cursor/selection	`actor`, `viewId`, `selection`, `cursor`
`Error`	Rejection or failure	`code`, `message`, optional `opBatchId`

5.2 REST API

The server exposes REST endpoints under the base path `/`.

5.2.1 Model state endpoints

Endpoint	Method	Purpose
`/models/\{modelId\}/snapshot`	GET	Get the current materialized snapshot of a model. Optional query parameter `?ref=<tagName>`.
`/models/\{modelId\}/rebuild`	POST	Rebuild materialized state from checkpoint plus complete tail, or full history. Transactional replacement.

Readable-model discovery uses `GET /models` and `GET /models/\{modelId\}/tags`. These filter by read access; ordinary plugin users do not need access to the admin catalog.

Example:

```
curl -s http://localhost:8081/models/demo/snapshot | jq .
```

Listing 16: Get snapshot

5.2.2 Admin endpoints

Endpoint	Method	Purpose
`/admin/models`	GET	List registered models (catalog).
`/admin/models/\{modelId\}`	POST	Register a new model.

	PUT	Rename with query parameter `modelName`.
	DELETE	Delete a model. Query param `?force=true` to override active-session guard.
`/admin/models/\{modelId\}/status`	GET	Aggregated status: snapshot counts, revision, consistency.
`/admin/models/\{modelId\}/rebuild-and-status`	POST	Rebuild and return status in one call.
`/admin/models/\{modelId\}/window?limit=25`	GET	Model window: status + recent activity + recent op batches.
`/admin/overview?limit=25`	GET	Overview for all known active/recent models.
`/admin/models/\{modelId\}/integrity`	GET	Integrity report: missing references, orphan nodes.
`/admin/models/\{modelId\}/acl`	GET, PUT	Read or update per-model ACL (admin, writer, reader user lists).
`/admin/models/\{modelId\}/tags`	POST	Create an immutable tag at the current HEAD.
`/admin/models/\{modelId\}/export`	GET	Export model package (snapshot, recovery data, tags and ACL).
`/admin/models/import?overwrite=true|false`	POST	Import a model package.
`/admin/models/\{modelId\}/checkpoints`	POST	Create immutable checkpoint.
`/admin/models/\{modelId\}/checkpoints/latest`	GET	Latest checkpoint summary.
`/admin/models/\{modelId\}/compact`	POST	Run compaction on the model (prune covered published payloads; supports dryRun).
`/admin/auth/diagnostics`	GET	Authorization diagnostics: resolved identity, subject, normalized roles.
`/admin/sessions`	GET	Active WebSocket sessions across all models.
`/admin/audit/config`	GET	Current audit configuration.

Checkpoint creation and inspection are model-admin actions. The latest summary returns model ID, revision, creation timestamp, SHA-256 hash and raw node/edge counts; no checkpoint gives 404. Compaction accepts `retainRevisions` and `dryRun=true|false` query parameters. Its report includes the verified checkpoint boundary, eligibility before the run, actual deletions and durable pruning progress. See Section 11 for the full maintenance workflow.

Examples:

```
# List all models
curl -s http://localhost:8081/admin/models | jq .

# Create a model
curl -s -X POST http://localhost:8081/admin/models/demo | jq .

# Get model status
curl -s http://localhost:8081/admin/models/demo/status | jq .

```

```
# Get model window
curl -s http://localhost:8081/admin/models/demo/window?limit=25 | jq .

# Get overview
curl -s http://localhost:8081/admin/overview?limit=25 | jq .

# Delete model
curl -s -X DELETE http://localhost:8081/admin/models/demo | jq .

# Force delete (override active-session guard)
curl -s -X DELETE "http://localhost:8081/admin/models/demo?force=true" | jq .
```

Listing 17: Admin API examples

5.2.3 Admin UI endpoint

Path `/admin-ui/`
Serves Static Svelte admin application (built from `admin-web/`).

When running in dev mode, the admin UI is served from `server/src/main/resources/META-INF/resources/admin-ui/` (populated by the build script). In production, the Quarkus application serves the built static assets.

6 Persistence

The Archimesh server persists all model state to Neo4j. This chapter describes the graph model, repository layer and key invariants.

6.1 Repository layer

The persistence layer is organized into a split repository with a main coordinator and specialized support classes:

`Neo4jRepositoryImpl` Main coordinating repository. Orchestrates op-log writes, materialized state mutations, head revision updates, snapshots, rebuilds, imports, exports, compactions and integrity checks.

`Neo4jOpLogSupport` Commit and operation persistence. Writes `:Commit` and `:Op` nodes, maintains the `NEXT` chain.

`Neo4jMaterializedStateSupport` Materialized graph mutation. Creates, updates and deletes elements, relationships, views, view objects, connections and folders. Applies LWW merges and tombstone checks. Maintains property clocks.

`Neo4jReadSupport` Snapshot export, checkout delta reads, consistency checks. Reads materialized state and converts it to the JSON snapshot/export format.

`Neo4jCompactionSupport` Bounded pruning of published operation payloads covered by a verified checkpoint and retention watermark. Retains acceptance receipts, clocks, and tombstones.

`Neo4jCheckpointSupport` Lossless raw graph capture, schema and checksum verification, and checkpoint restoration.

6.2 Graph model

The database stores these kinds of state per model:

1. Model catalog and ACL metadata
2. Materialized current state
3. Committed op-log history
4. Merge metadata (tombstones, property clocks)
5. Immutable recovery checkpoints and named tag snapshots
6. Durable publication intents and retained acceptance receipts

All identity is model-scoped: materialized entity identity is `(modelId, id)`; commit identity is `(modelId, opBatchId)`; tag identity is `(modelId, tagName)`.

6.2.1 Core model node

Every model is rooted at a `:Model` node:

```
{
  modelId, modelName,
  registered: true,
  headRevision: <number>,
  createdAt, updatedAt,
  adminUsers: ["user1", ...],
  writerUsers: ["user2", ...],
```

```
  readerUsers: ["user3", ...]
}
```

Listing 18: Model node properties

ACL data is stored directly on `:Model` as user-id arrays.

6.2.2 Materialized state nodes

Label	Properties
`:Element`	`modelId`, `id`, `archimateType`, `name`, `documentation`, `name_lamport`, `name_clientId`, `documentation_lamport`, `documentation_clientId`
`:Relationship`	`modelId`, `id`, `archimateType`, `name`, `documentation`, `sourceId`, `targetId`, plus LWW clocks for name, documentation, source and target.
`:Folder`	`modelId`, `id`, `folderType`, `name`, `documentation`, plus LWW clocks.
`:View`	`modelId`, `id`, `name`, `documentation`, `notationJson`, plus LWW clocks.
`:ViewObject`	`modelId`, `id`, `notationJson`, plus per-field LWW clocks for geometry, style and label fields.
`:Connection`	`modelId`, `id`, `sourceViewObjectId`, `targetViewObjectId`, `representsId`, `notationJson`, plus per-field LWW clocks.

6.2.3 Materialized relationships

```
(:Model)-[:HAS_ELEMENT]->(:Element)
(:Model)-[:HAS_REL]->(:Relationship)
(:Relationship)-[:SOURCE]->(:Element)
(:Relationship)-[:TARGET]->(:Element)

(:Model)-[:HAS_FOLDER]->(:Folder)
(:Folder)-[:HAS_FOLDER]->(:Folder)
(:Folder)-[:CONTAINS_ELEMENT]->(:Element)
(:Folder)-[:CONTAINS_REL]->(:Relationship)
(:Folder)-[:CONTAINS_VIEW]->(:View)

(:Model)-[:HAS_VIEW]->(:View)
(:View)-[:CONTAINS]->(:ViewObject)
(:ViewObject)-[:REPRESENTS]->(:Element)
(:ViewObject)-[:CHILD_MEMBER]->(:ViewObject)

(:View)-[:CONTAINS]->(:Connection)
(:Connection)-[:REPRESENTS]->(:Relationship)
(:Connection)-[:FROM]->(:ViewObject)
(:Connection)-[:TO]->(:ViewObject)
```

Listing 19: Core relationships

6.2.4 Folder rules

- Root folders use stable synthetic ids: `folder:root-strategy`, `folder:root-business`, `folder:root-diagrams`, etc.
- User folders use immutable ids — rename changes the name property but not the node identity.

- A model-tree object belongs to exactly one folder at a time.
- Cross-root folder moves are rejected by validation.
- Non-empty folder delete is rejected.

6.2.5 Op-log history

Commit nodes preserve the accepted timeline. Linked operation payloads remain until checkpoint-covered pruning:

```
(:Commit {
  modelId, opBatchId,
  revisionFrom, revisionTo,
  ts, opCount,
  published, payloadPruned, batchJson
})

(:Op {
  seq, type, targetId, payloadJson
})
```

Listing 20: Op-log nodes

Relationships:

```
(:Model)-[:HAS_COMMIT]->(:Commit)
(:Commit)-[:HAS_OP]->(:Op)
(:Commit)-[:NEXT]->(:Commit)
```

Listing 21: Op-log relationships

- `revisionFrom..revisionTo` is the assigned revision range for one committed submit batch.
- `payloadJson` stores the normalized op payload.
- `seq` preserves op order inside a batch.
- `NEXT` keeps the timeline linear.
- Production atomic commits store `batchJson` and `published=false` together with projection and head changes. Broker acknowledgment permits setting `published=true`.
- Pruning removes `Op` nodes and `batchJson`, then marks `payloadPruned=true`. The commit remains as an acceptance receipt.

6.2.6 Recovery checkpoints

```
(:Model)-[:HAS_CHECKPOINT]->(:ModelCheckpoint {
  modelId, revision, sha256, payloadJson
})
```

Listing 22: Checkpoint node and ownership

`payloadJson` contains the versioned checkpoint document, including creation timestamp, raw nodes and edges, all merge metadata and the checksum. Uniqueness is `(modelId, revision)`. At an existing boundary, creation returns the immutable checkpoint rather than replacing it.

The checkpoint's projection does not duplicate catalog metadata, ACLs, tags, commits or its own storage node. Its reserved model endpoint is rebound to the destination model on restore. A checkpoint is richer than a client snapshot; it includes raw clock and orphan safety state needed for deterministic merges. See Section 11.2 for validation and recovery requirements.

6.2.7 Tags

Tags are stored as standalone nodes:

```
(:ModelTag {
  modelId, tagName, description,
  revision, createdAt, snapshotJson
})
```

Listing 23: Tag node

- Tags are immutable — once created, they cannot be modified or deleted (tag deletion is disabled by default).
- `snapshotJson` stores the materialized snapshot at the tagged revision.
- Tags do not create branches; they are read-only pointers on the linear timeline.

6.2.8 Merge metadata — tombstones

Tombstones prevent stale recreates after deletes:

```
(:ElementTombstone { modelId, id, lamport, clientId })
(:RelationshipTombstone { modelId, id, lamport, clientId })
(:ViewTombstone { modelId, id, lamport, clientId })
(:ViewObjectTombstone { modelId, id, lamport, clientId })
(:ConnectionTombstone { modelId, id, lamport, clientId })
```

Listing 24: Tombstone nodes

Tombstones are anchored from `:Model` via specific relationship types (`HAS_ELEMENT_TOMBSTONE`, etc.). Compaction reports tombstone eligibility but retains them for delete safety.

6.2.9 Merge metadata — property clocks

Property merge state is stored in dedicated metadata nodes:

```
(:PropertyClock {
  modelId, targetId, key,
  lamport, clientId, deleted
})
```

Listing 25: PropertyClock node

Anchored via `(:Model)-[:HAS_PROPERTY_CLOCK]->(:PropertyClock)`. Supports LWW and OR-set style property semantics. Compaction retains all property clocks, including orphan safety metadata.

6.3 Neo4j constraints

The schema file at `neo4j/schema.cypher` defines the following uniqueness constraints:

```
(:Model {modelId})                    UNIQUE
(:Element {modelId, id})              UNIQUE
(:Relationship {modelId, id})         UNIQUE
(:View {modelId, id})                 UNIQUE
(:Folder {modelId, id})               UNIQUE
(:ViewObject {modelId, id})           UNIQUE
(:Connection {modelId, id})           UNIQUE
```

```
(:Commit {modelId, opBatchId})        UNIQUE
(:ModelTag {modelId, tagName})        UNIQUE
(:ModelCheckpoint {modelId, revision}) UNIQUE
```

Listing 26: Uniqueness constraints

An index on `(:Commit)` by `(modelId, revisionFrom, revisionTo)` supports efficient delta window queries.

6.4 Persistence invariants

- Materialized entity identity is `(modelId, id)` — never rely on node-internal ids across queries.
- Commit/idempotency identity is `(modelId, opBatchId)` — duplicate submission of the same batch is a no-op.
- Atomic model commits persist log, projection, head and delivery intent together. A failed transaction changes none of them.
- Recovery requires a verified checkpoint and complete tail, or full log from revision 1. Replacement is transactional. Pruning keeps acceptance receipts and all CRDT safety metadata.
- Repository write paths must fail fast on Neo4j persistence errors — never silently skip a mutation.
- Notation field definitions must flow through `NotationMetadata`, the shared source of truth for supported notation fields and persisted merge metadata.
- Compaction never allows stale `Create*` or `Update*` to beat a logically newer delete — tombstones are retained.

6.5 Configuration

Property	Env var	Default
`app.neo4j.uri`	`NEO4J_URI`	`bolt://localhost:7687`
`app.neo4j.username`	`NEO4J_USER`	`neo4j`
`app.neo4j.password`	`NEO4J_PASSWORD`	`devpassword`

In production, always set `NEO4J_PASSWORD` to a strong value. The default `devpassword` is for local development only.

7 Messaging

Kafka is used as the ordered fan-out path for accepted Archimesh operation batches, lock events and presence events.

7.1 Topic layout

All models share three topics. Partition use is independent of model count:

Topic	Content	Record key
`archimesh.model.ops`	Committed op batches	`modelId`
`archimesh.model.locks`	Lock events	`modelId`
`archimesh.model.presence`	Cursor/selection broadcasts	`modelId`

The prefix is configurable via `KAFKA_TOPIC_PREFIX` (default `archimesh.model`). One partition per topic gives three application partitions total, plus any broker-internal partitions. Adding a model does not create topics or partitions. This layout replaces the old per-model topics without compatibility routing; use a fresh Kafka start.

7.2 Keyed ordering and capacity

Every record uses `modelId` as its key, including locks and presence. With stable partition counts, key partitioning keeps a model's records in one partition per topic. Models can therefore share partitions while preserving their own order within each event kind. The durable dispatcher publishes operation batches in model revision order. There is no ordering guarantee between models or between the three topics.

One partition per topic is sufficient for local development. Larger fixed counts may be provisioned initially for throughput. Increasing counts during live publication can move a key to a new partition before old records have been consumed; drain traffic and consumers and coordinate a cutover before resizing. Busy models share partition capacity with other models, and retention and broker ACLs apply to whole topics. Application model ACLs continue to govern client subscriptions.

7.3 Producer

The `KafkaPublisherImpl` uses a `KafkaProducer<String,String>` with:

- `acks=all` — wait for all in-sync replicas
- `retries=3`
- `delivery.timeout.ms=120000`
- String serializers for both key and value

Publishing is asynchronous. The `publishOps` method returns a `CompletableFuture<Void>` that completes when the broker acknowledges the send. Committed operation batches are published by `CommitDeliveryService`. It reads pending Neo4j payloads in model revision order, waits for broker acknowledgment, then marks publication complete. Failed or interrupted sends leave the durable intent pending for retry.

Pending model discovery is separate from payload decoding. A cursor rotates through sorted pending model IDs, scanning at most 100 IDs per pass. A bounded pool handles independent models concurrently, with at most one queued or running worker per model. Each visit publishes a limited prefix in revision order, awaiting broker acknowledgment and the Neo4j flag write before its successor. A large backlog in one model cannot fill the discovery window for every other model.

The two branches in Figure 3 have separate durability boundaries. Acceptance confirms the Neo4j commit to the submitting session; broadcast reaches every subscriber through the Kafka consumer. Either can arrive first. A failed model transaction receives no acceptance.

7.4 Consumer

The `KafkaConsumerImpl` runs a single-threaded consume loop with:

- Group id: `archimesh-server` (configurable via `KAFKA_CONSUMER_GROUP_ID`)
- Auto offset reset: `earliest`
- Exact subscription to the three `<prefix>.<kind>` topics
- 500 ms poll timeout

On receiving a record, the consumer derives the event kind from the topic and the model id from the record key. Missing or blank keys and conflicting payload model ids are rejected. Lock and presence payloads need not carry a model id. The consumer broadcasts only to sessions registered for the keyed model:

ops Broadcasts `OpsBroadcast` to all sessions for the model. The submitter reconciles its own echo against the optimistic local state.
locks Broadcasts `LockEvent`.
presence Broadcasts `PresenceBroadcast`.

The consumer is resilient to Kafka unavailability: on connection failure it sleeps and retries. While publication or consumption is unavailable, other clients can lag behind the accepted state. Durable publication retry and a subsequent complete delta or snapshot checkout restore synchronization.

7.5 Retention

ops Retain enough for expected consumer outages and lag. Neo4j checkpoints plus the operation tail provide the implemented rebuild source; Kafka is the fan-out path, not the model backup. Keep delete-based retention: compaction by model key would remove earlier operation batches still needed by lagging consumers.
locks Short retention (minutes). Lock state is ephemeral.
presence Very short retention (minutes). Presence is transient.

7.6 Publication failure and duplicate delivery

If Kafka accepts a batch but the server stops before Neo4j records the publication flag, restart can publish it again. Delivery is at least once; clients tolerate duplicate committed ranges and reconcile accepted echoes. Retrying a submitted batch consults the persisted acceptance receipt and returns the original revision range without committing again.

Publication payloads are retained until acknowledged. Pruning never crosses the first unpublished or unclassified legacy batch, even when a newer checkpoint covers its state. Lock and presence events use direct Kafka publication and do not share the durable operation outbox.

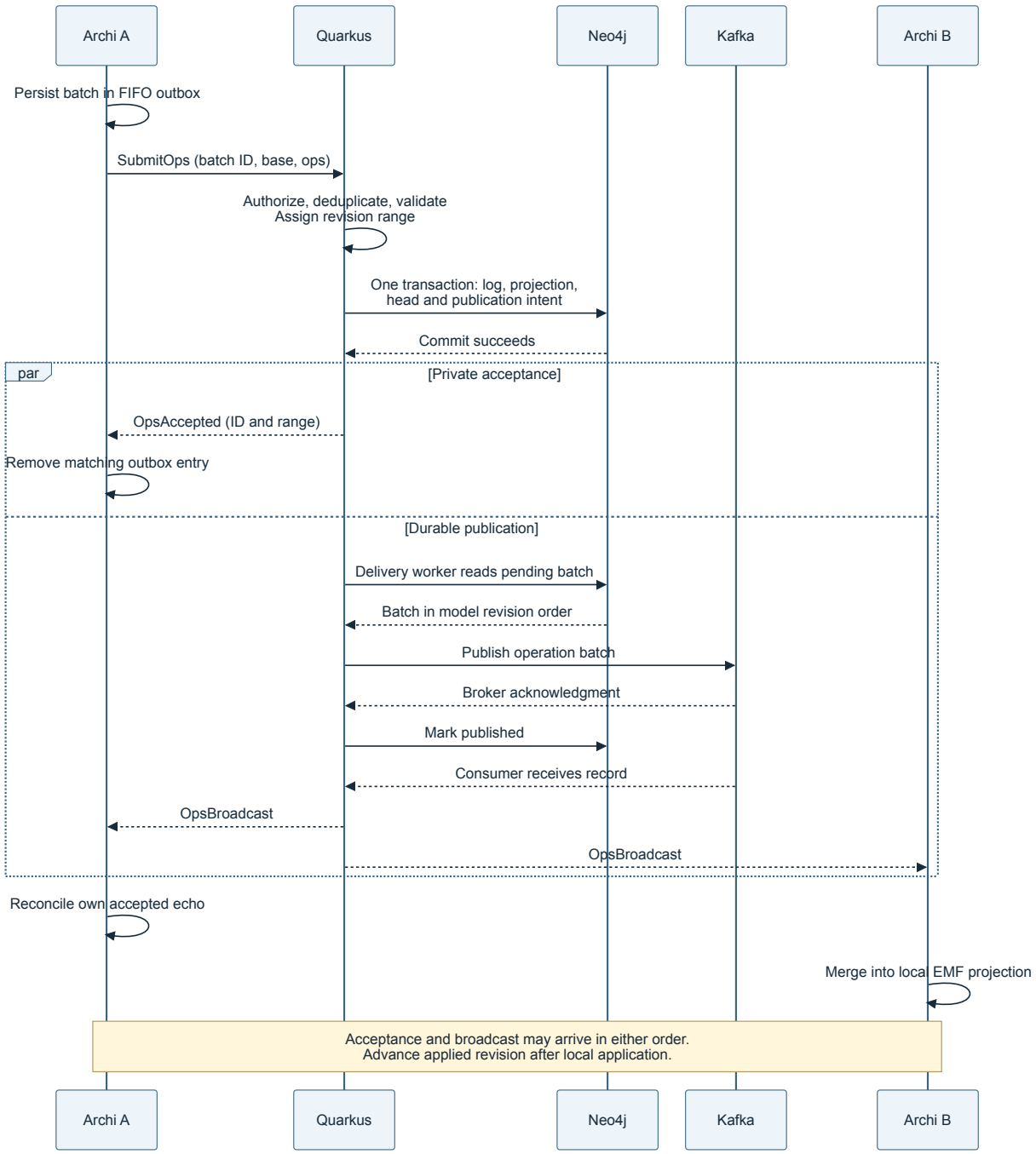


Figure 3: Durable acceptance and asynchronous operation fan-out.

A publication failure, invalid or missing payload, or acknowledgment-write failure stops that model's visit and defers retries for at least one second. Its batch and successors remain pending while other model workers continue. Stored payload identity, range and operation count must match the receipt. A slow broker send retains its model slot until it settles; later batches cannot overtake it. If every worker slot is occupied, other models wait. Lease renewal runs separately from the model workers, and shutdown interrupts pending waits without acknowledging them. Cursor and cooldown state are process-local; restart recovers work from the durable queue.

7.7 Configuration

Property	Env var	Default
`kafka.bootstrap.servers`	`KAFKA_BOOTSTRAP_SERVERS`	`localhost:9092`
`kafka.topic.prefix`	`KAFKA_TOPIC_PREFIX`	`archimesh.model`
`kafka.consumer.group-id`	`KAFKA_CONSUMER_GROUP_ID`	`archimesh-server`
`kafka.consumer.auto-offset-reset`	`KAFKA_CONSUMER_AUTO_OFFSET_RESET`	`earliest`

Delivery limits are configured separately from Kafka:

`app.delivery.concurrent-models` `ARCHIMESH_DELIVERY_CONCURRENT_MODELS`, default 4.

`app.delivery.batches-per-model` `ARCHIMESH_DELIVERY_BATCHES_PER_MODEL`, default 25.

Both must be positive. Submit bursts coalesce immediate scan requests; the periodic scan also runs once per second. Apply `neo4j/schema.cypher` to install the `commit_delivery` queue index on `(published, modelId, revisionFrom)`. Model-specific delivery warnings identify blocked models; broker-wide and database-wide outages can still affect all workers.

8 CRDT and Merge Semantics

Archimesh uses a hybrid consistency model: the server is the single point of serialization for revision ordering, but per-field CRDT merges handle concurrent edits on different fields of the same object without requiring explicit locks.

8.1 Causal identity

Every operation carries a `causal` object:

```
{
  "clientId": "<stable client/session id>",
  "lamport": <monotonic per-client counter>,
  "opId": "<idempotency identity>"
}
```

Listing 27: Causal metadata

If the client omits `causal`, the server fills it deterministically from the assigned revision range. The comparison rule for Last-Writer-Wins:

> `(lamportA, clientIdA)` wins over `(lamportB, clientIdB)` iff `lamportA > lamportB`, or `lamportA == lamportB` and `clientIdA >= clientIdB` (lexicographic).

8.2 Merge types by domain

8.2.1 Semantic element fields

Fields: `name`, `documentation`.

- Type: LWW Register per field per element.
- Metadata: field clock map `elementId -> \{ field -> (lamport, clientId) \}`.
- Merge: apply incoming field if the incoming tuple wins the field clock; otherwise ignore.

8.2.2 Relationship fields

Fields: `name`, `documentation`, `sourceId`, `targetId`.

Same LWW register semantics as elements, with per-field clocks stored on the `:Relationship` node.

8.2.3 Diagram notation fields

View objects and connections use per-field LWW registers for:

View objects `x`, `y`, `width`, `height`, `type`, `alpha`, `lineAlpha`, `lineWidth`, `lineStyle`, `textAlignment`, `textPosition`, `gradient`, `iconVisibleState`, `deriveElementLineColor`, `fillColor`, `lineColor`, `font`, `fontColor`, `iconColor`, `imagePath`, `imagePosition`, `name`, `documentation`.

Connections `type`, `nameVisible`, `textAlignment`, `textPosition`, `lineWidth`, `name`, `lineColor`, `font`, `fontColor`, `documentation`, `bendpoints`.

Per-field notation clocks are stored as properties on the `:ViewObject` and `:Connection` nodes (e.g. `notation_lamport_x`, `notation_clientId_x`).

8.2.4 View object child membership

Type: OR-set. Operations: `AddViewObjectChildMember`, `RemoveViewObjectChildMember`.

Member clocks are stored via `:PropertyClock` nodes in the `VIEWOBJECT_CHILD` namespace. Materialized as `(:ViewObject)-[:CHILD_MEMBER]->(:ViewObject)` edges.

8.2.5 Property set membership

Type: OR-set. Operations: `AddPropertySetMember`, `RemovePropertySetMember`.

Member clocks are stored via `:PropertyClock` nodes. Materialized as canonical JSON string arrays in the snapshot/export output.

8.3 Delete semantics — tombstones

When an entity is deleted, a tombstone is recorded with the deleting operation's causal clock:

```
1 { modelId, id, lamport, clientId }
```

Listing 28: Tombstone record

Rules:

- `Delete*` always records or updates the tombstone (even if the entity was already missing).
- Stale `Update*` or `Create*` (with a causal clock older than the tombstone) is ignored — preventing resurrection.
- A newer `Create*` (with a causal clock that beats the tombstone) may clear the tombstone and recreate the entity.

Tombstones persist across compactions. They are *reported* as eligible under the watermark but *retained* for delete safety.

8.4 Notation field validation

The server validates notation field keys against a whitelist defined in `NotationMetadata`. Operations containing unknown notation keys are rejected with `PRECONDITION_FAILED`. This prevents clients from emitting fields the server cannot merge deterministically.

The whitelist covers all currently supported notation fields for view objects and connections. The notation inventory is kept in parity across:

- Server runtime whitelist constants (`ArchimeshService`).
- JSON schema definitions (`schemas/ops.json`).
- Client serializer/deserializer field coverage.

- Client inventory test expectations (`OpMapperNotationInventoryTest`).

The script `scripts/check-notation-parity.sh` verifies this parity and is wired into the preflight hardening checks.

8.5 Lock model

Lease-based locks are used for notation-sensitive edits:

- Operations requiring locks: `UpdateViewObjectOpaque`, `UpdateConnectionOpaque`.
- Lock targets are the affected view object or connection ids.
- Default TTL: 10 seconds.
- A client must hold the lock for a target before submitting an opaque notation update for that target.
- If another client holds a conflicting lock, the submit is rejected with `LOCK_CONFLICT`.
- Locks are automatically released on WebSocket close.
- Locks are advisory, not enforced by the merge layer — they reduce destructive interleaving on noisy diagram edits while keeping semantic collaboration workable.

8.6 CRDT maturity

The CRDT implementation covers the currently supported operation set at the P0 and P1 levels described in `crdt-spec.md`. Remaining work includes:

- Extending collection-aware operations to additional domains beyond property sets and view object children.
- Gathering long-running workload evidence that metadata growth remains operationally bounded under production-like usage.

Any remaining non-CRDT domains are explicitly marked as unsupported rather than left as implicit gaps.

9 Authorization

Authorization is enforced in Quarkus using a server-side Policy Enforcement Point (PEP) and a project-specific in-process Policy Decision Point (PDP).

9.1 Identity modes

Identity resolution is pluggable. The mode is selected via `ARCHIMESH_IDENTITY_MODE` (or `app.identity.mode`):

`bootstrap` Local/dev mode. No external identity provider.

- REST: identity from `X-Archimesh-User` and `X-Archimesh-Roles` headers.
- WebSocket: identity from query parameters `user` and `roles`.
- Intended for local development and simple standalone deployments.

`proxy` Trusted forwarded headers from a reverse proxy.

- REST: identity from trusted forwarded headers (default `X-Forwarded-User`, `X-Forwarded-Roles`).
- WebSocket: identity from trusted forwarded headers captured during the handshake.
- Header names are configurable via `app.identity.proxy.user-header` and `app.identity.proxy.roles-header`.
- Do not expose `proxy` mode directly to untrusted clients without a trusted proxy in front.

`oidc` Direct principal/role resolution through Quarkus auth.

- REST: identity from the authenticated Quarkus `SecurityContext`.
- WebSocket: identity from the authenticated WebSocket upgrade principal and role checks captured during the handshake.
- Can use local JWT verification or an external OIDC provider.
- No reverse proxy required in this mode.

9.2 Role model

Role names from the identity provider are normalized into three canonical internal roles:

`admin` Catalog-wide administration: register/delete models, manage ACLs, run compactions, export/import, view diagnostics.

`model_reader` Read model snapshots and join read-only sessions.

`model_writer` Submit operations, acquire locks, send presence. Implies `model_reader`.

Role names are configurable:

```
# Default canonical role names
app.authz.admin-role=admin
app.authz.reader-role=model_reader
app.authz.writer-role=model_writer

# Optional provider-specific aliases (comma-separated)
app.authz.admin-role-aliases=realm-admin,archimesh-admin
app.authz.reader-role-aliases=realm-viewer,archimesh-reader
app.authz.writer-role-aliases=realm-editor,archimesh-writer
```

Listing 29: Role configuration

Aliases normalize provider-specific group names (e.g. Keycloak realm roles, Auth0 permissions) into the canonical internal roles before the PDP evaluates policy.

9.3 Policy model

The PDP decides on these actions:

- Admin catalog actions (list models, create, delete)
- Model join
- Snapshot read
- Submit operations
- Lock acquire/release
- Presence broadcast
- Tag creation
- ACL management
- Export/import
- Compaction

Authorization is deny-by-default when `ARCHIMESH_AUTHZ_ENABLED=true`.

9.3.1 Model-scoped ACLs

Per-model ACLs are stored on the `:Model` node as `adminUsers`, `writerUsers` and `readerUsers` arrays. When an ACL is configured for a model, it overrides the generic reader/writer role checks for that model:

- A user listed in `adminUsers` gets admin access to that model.
- A user listed in `writerUsers` can submit ops and lock for that model.
- A user listed in `readerUsers` can read snapshots for that model.
- Catalog-wide admin actions still require the global `admin` role.

9.4 JWT and OIDC setup

9.4.1 Local JWT (dev mode)

For local development with bearer tokens, use `app.identity.mode=oidc` with Quarkus JWT validation:

```
app.authz.enabled=true
app.identity.mode=oidc
mp.jwt.verify.publickey.location=src/test/resources/jwt/publicKey.pem
mp.jwt.verify.issuer=https://archimesh.dev
```

Listing 30: Local JWT config

Mint tokens with the dev script:

```
./scripts/mint-dev-jwt.sh --user alice --roles admin,model_writer,model_reader
```

Listing 31: Mint dev JWT

Use the token in requests:

```
TOKEN=$(./scripts/mint-dev-jwt.sh --user alice --roles admin)
curl -s -H "Authorization: Bearer $TOKEN" \
  http://localhost:8081/admin/models | jq .
```

Listing 32: Bearer token usage

9.4.2 External OIDC

For production with an external provider (Keycloak, Auth0):

```
app.authz.enabled=true
app.identity.mode=oidc
quarkus.oidc.enabled=true
quarkus.oidc.auth-server-url=https://idp.example/realms/archimesh
quarkus.oidc.client-id=archimesh-server
quarkus.oidc.application-type=service
quarkus.oidc.roles.role-claim-path=groups
```

Listing 33: External OIDC config

Concrete provider examples are in `server/examples/`.

9.5 Authorization diagnostics

The endpoint `GET /admin/auth/diagnostics` returns the resolved identity mode, subject, normalized roles and authorization status for the current request. The admin UI exposes this through the **Auth Check** action. This is useful for debugging misconfigured role mappings or proxy headers.

9.6 Configuration summary

Property	Env var	Default
app.authz.enabled	ARCHIMESH_AUTHZ_ENABLED	false
app.identity.mode	ARCHIMESH_IDENTITY_MODE	bootstrap
app.identity.proxy.user-header	ARCHIMESH_PROXY_USER_HEADER	X-Forwarded-User
app.identity.proxy.roles-header	ARCHIMESH_PROXY_ROLES_HEADER	X-Forwarded-Roles
app.authz.admin-role	ARCHIMESH_AUTHZ_ADMIN_ROLE	admin
app.authz.reader-role	ARCHIMESH_AUTHZ_READER_ROLE	model_reader
app.authz.writer-role	ARCHIMESH_AUTHZ_WRITER_ROLE	model_writer
quarkus.oidc.enabled	ARCHIMESH_QUARKUS_OIDC_ENABLED	false

10 Administration and Supervision

This chapter covers day-to-day operational supervision: the admin UI, audit logging, activity monitoring and session inspection.

10.1 Admin UI

The admin UI is a Svelte application served by Quarkus at `/admin-ui/`. It is organized into seven routes:

Overview Dashboard showing all known models with status summaries, head revisions, counts of elements, relationships and views, and active sessions. Colour-coded status: green (no issues), yellow (stale), red (integrity problems or consistency check failures).
Models Detailed per-model view: catalog metadata, ACL configuration, head revision, estimated counts, tag list and available actions (create tag, rebuild, checkpoint, preview pruning, prune, export, delete). Maintenance shows the latest checkpoint and each pruning result, including eligibility, actual payload deletions and durable progress. The model selector persists the chosen model in browser storage.
Versions Tag management: list existing tags with description, revision and creation timestamp. Create new tags from current HEAD. Export and import model packages.
Access Identity and authorization: view the current resolved identity, normalized roles, and per-model ACL configuration. Update ACLs for a model. Run authorization diagnostics.
Sessions Live session inspection: active WebSocket sessions per model, with user id, session id, connected-since timestamp and held locks.
Operations Recent model activity: operator-facing events emitted by the server for the selected model (join, disconnect, submit, accept, reject, lock, rebuild, compact, tag, export, import). Recent op batches with batch id, revision range, op count and timestamp.
Audit Admin and WebSocket audit trail: structured log events with resolved user id, action, request context and timestamp. Websocket lifecycle events (open rejected, join, message rejected, close).

The admin UI supports:

- Bootstrap auth inputs for `X-Archimesh-User` and `X-Archimesh-Roles`, plus an optional bearer token for OIDC mode.
- Configurable poll interval (persisted in browser storage; set to 0 to disable auto-refresh).
- When a bearer token is present, admin 401/403 failures explain likely token-expiry or missing-role/model-access problems.

10.2 Audit logging

The server emits two structured audit log streams:

10.2.1 admin_audit

Emitted for admin diagnostic reads, model window reads and mutating admin actions. Each event is an INFO-level log line with a stable JSON payload containing:

- Resolved user id
- Action name
- Request context (model id, endpoint, parameters)

- Timestamp

Rejected admin mutations (409, 400) are audited too.

Operational guidance:

- Ship these JSON lines to a dedicated audit sink or index if relied on for operator review.
- Keep retention shorter than ordinary application logs if dashboard polling is frequent.
- Avoid over-aggressive dashboard refresh intervals in production — repeated window reads produce repeated audit events.
- Treat `admin_audit` as machine-readable output; parse the JSON payload, do not depend on free-form message text.

10.2.2 ws_audit

Emitted for WebSocket lifecycle events:

- Rejected opens
- Successful joins
- Rejected WebSocket messages
- Closes

Configurable via `ARCHIMESH_AUDIT_WEBSOCKET_ACTIONS` (comma-separated action names). Set to an empty value to disable WebSocket audit entirely.

Set `ARCHIMESH_AUDIT_WEBSOCKET_VERBOSE=true` to also emit `WebSocketMessageAccepted` events for accepted `SubmitOps`, `AcquireLock`, `ReleaseLock` and `Presence` messages.

10.3 Supervision checklist

Regular operational supervision tasks:

- Check the admin overview for models with stale head revisions (clients not submitting) or integrity issues.
- Check the sessions page for unexpected disconnects or abandoned sessions.
- Check the operations page for repeated `SubmitOpsRejected*` events — these indicate client problems.
- Check the audit page for unauthorized access attempts.
- Monitor Neo4j disk usage — op-log and materialized state grow with model activity. Payload pruning retains receipts and all CRDT metadata.
- Inspect the latest checkpoint and preview the selected retention policy before pruning. Archive full history before the first checkpoint if required.
- If scheduled maintenance is enabled, review its result/failure logs, checkpoint age and retained checkpoint count. Configuration is documented in Section 11.4.
- Monitor Kafka topic sizes and consumer lag.
- Periodically run `GET /admin/models/\{modelId\}/integrity` and check for orphan nodes or missing references.

11 System Management

This chapter covers system management tasks: creating and deleting models, checkpoints, payload pruning, scheduled maintenance, exports, imports, rebuilds, tags and integrity checks. API examples assume local development; add the identity headers or bearer token required by your deployment.

11.1 Model lifecycle

11.1.1 Creating a model

Models must be explicitly registered before clients can use them:

```
curl -s -X POST http://localhost:8081/admin/models/demo | jq .
```

Listing 34: Register a model

Optionally provide a display name:

```
curl -s -X POST http://localhost:8081/admin/models/demo \
  --get --data-urlencode 'modelName=Demo Model' | jq .
```

Listing 35: Register with name

The creating user (resolved from the current identity) is seeded as the initial model admin, writer and reader in the ACL.

11.1.2 Deleting a model

```
curl -s -X DELETE http://localhost:8081/admin/models/demo | jq .
```

Listing 36: Delete a model

Deletion is refused while the model has active sessions. To override:

```
curl -s -X DELETE "http://localhost:8081/admin/models/demo?force=true" | jq .
```

Listing 37: Force delete

11.2 Recovery checkpoints

A checkpoint is a durable recovery base at a committed revision C. It captures the raw graph projection: elements, relationships, folders, views, diagram objects, connections, their edges and properties, all field and membership clocks, property clocks, and tombstones. Orphan safety metadata is included. Catalog metadata, ACLs, tags and acceptance receipts are stored separately.

The format `archimesh-checkpoint-v1` records model ID, revision, creation timestamp, raw state and a SHA-256 checksum over canonical JSON. Creation verifies that head and committed history agree and that receipt and recovery coverage are complete. Reading or restoring verifies checksum, scope, node identities, supported property types, labels and edge endpoints. Unsupported graph state fails creation before it can authorize pruning. The checksum detects accidental changes; it is not an authenticity signature.

```
curl -s -X POST http://localhost:8081/admin/models/demo/checkpoints | jq .
curl -s http://localhost:8081/admin/models/demo/checkpoints/latest | jq .
```

Listing 38: Create and inspect a checkpoint

Creation and inspection require model-admin access when authorization is enabled; global admins also have access. Inspection returns 404 when no checkpoint exists. Creating again at the same model/revision returns the existing immutable checkpoint. Tags serve a different purpose: they name historical client-facing snapshots and remain readable after payload pruning.

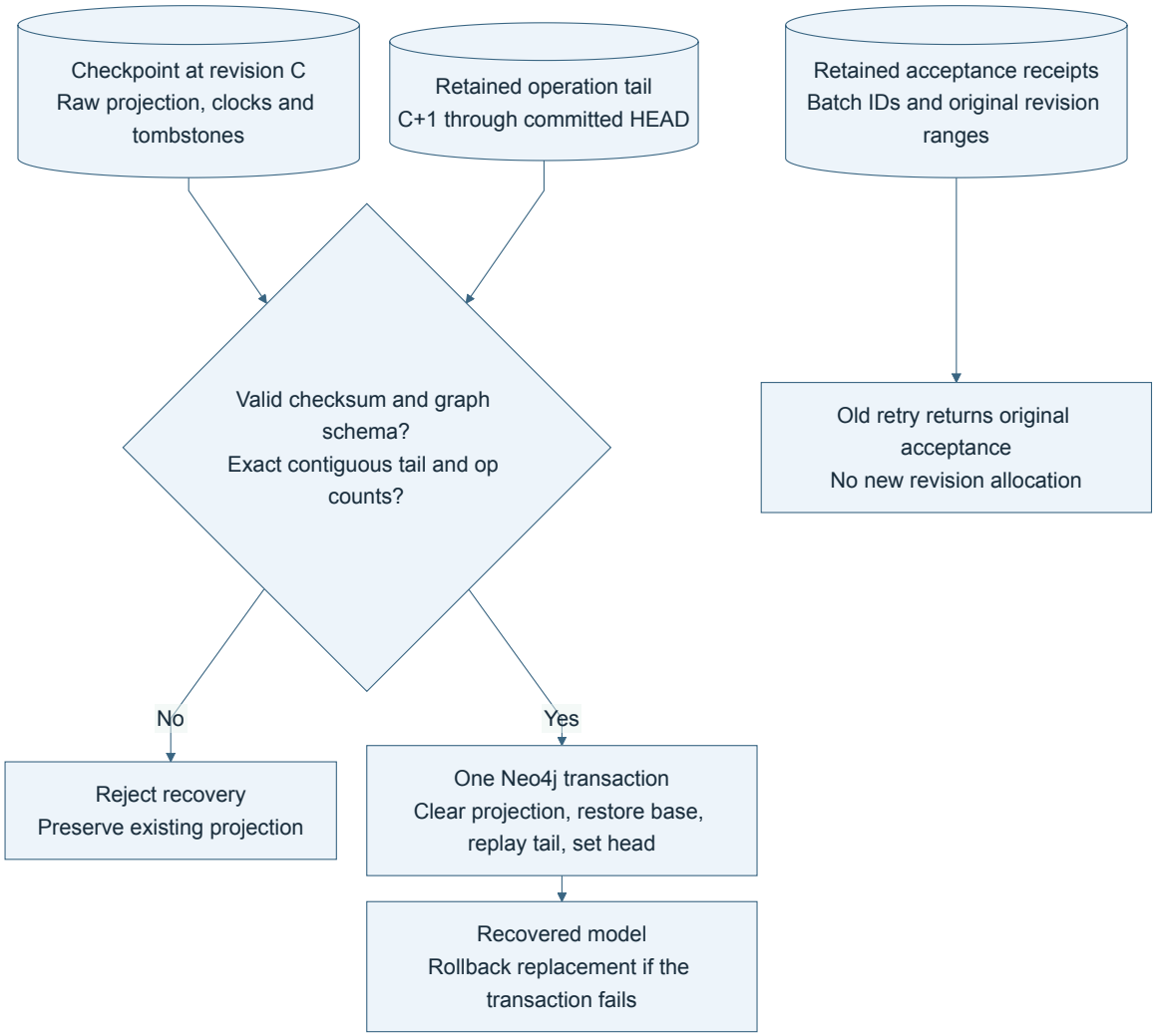


Figure 4: Recovery from a checkpoint and complete tail; receipts preserve retry identity.

11.3 Compaction and payload pruning

Compaction removes checkpoint-covered, published operation payloads below a configurable retention watermark. Acceptance receipts, field and membership clocks, property clocks (including orphans), and tombstones remain.

11.3.1 Running compaction

```
# Archive full history before the first checkpoint if required.
curl -s http://localhost:8081/admin/models/demo/export -o demo-archive.json
curl -s -X POST http://localhost:8081/admin/models/demo/checkpoints
curl -s http://localhost:8081/admin/models/demo/checkpoints/latest
```

```
curl -s -X POST \
  'http://localhost:8081/admin/models/demo/compact?retainRevisions=1000&dryRun=true'
curl -s -X POST \
  'http://localhost:8081/admin/models/demo/compact?retainRevisions=1000'
```

Listing 39: Checkpoint, preview, and prune a model

The response reports `checkpointRevision`, `watermarkRevision`, `eligibleBatchCount` before the run, `prunedBatchCount`, `deletedOpCount`, `prunedThroughRevision`, `dryRun`, and an explanatory `note`. Preview returns zero deletion counts. `eligibleBatchCount` is measured before the run. Repeat applied calls until it reaches zero; a run can finish its selected batches while a larger backlog still remains.

11.3.2 Compaction safety

- Without a verified checkpoint, no payloads are removed.
- Only whole batches ending at or below both the checkpoint and `latestCommitRevision - retainRevisions` are eligible.
- The first unpublished (or unclassified legacy) batch fences later pruning. Head/commit disagreement disables pruning.
- Each applied call handles at most `app.compaction.max-batches-per-run` batches (default 100). Repeat until no eligible batches remain.
- Commit acceptance receipts remain; only Op nodes and stored batch payloads are removed. All clocks and tombstones remain for stale-write/delete safety.
- Archive full history before creating the first checkpoint if individual old operations are needed: checkpointed exports omit the covered prefix payloads even if those payloads have not yet been pruned. Checkpoints share the model database and do not replace off-system backups.

Checkpoint capture and recovery use memory proportional to the model and run under the existing service serialization; large models can pause edits. The deletion limit bounds batches, not bytes or transaction duration. A large single batch can still be expensive. Receipts and all CRDT metadata remain; metadata garbage collection is not implemented.

11.4 Optional scheduled maintenance

Scheduled maintenance is disabled by default. Enabling it applies the policy to all registered models and does not create an external archive. The first pass starts after the configured interval. One background thread rotates through a bounded set of models from the sorted catalog, resuming from durable model state on subsequent passes. Rotation restarts from the first model after server restart.

Property suffix	Environment variable and default
`enabled`	`ARCHIMESH_CHECKPOINT_MAINTENANCE_ENABLED` = `false`
`interval-seconds`	`ARCHIMESH_CHECKPOINT_INTERVAL_SECONDS` = `300`
`min-revisions`	`ARCHIMESH_CHECKPOINT_MIN_REVISIONS` = `1000`
`retain-checkpoints`	`ARCHIMESH_CHECKPOINT_RETAIN_COUNT` = `2`
`models-per-run`	`ARCHIMESH_CHECKPOINT_MODELS_PER_RUN` = `10`

The property prefix for this table is `app.checkpoints.maintenance.`. The interval is a delay in seconds between completed passes. The revision threshold counts accepted revisions since the latest

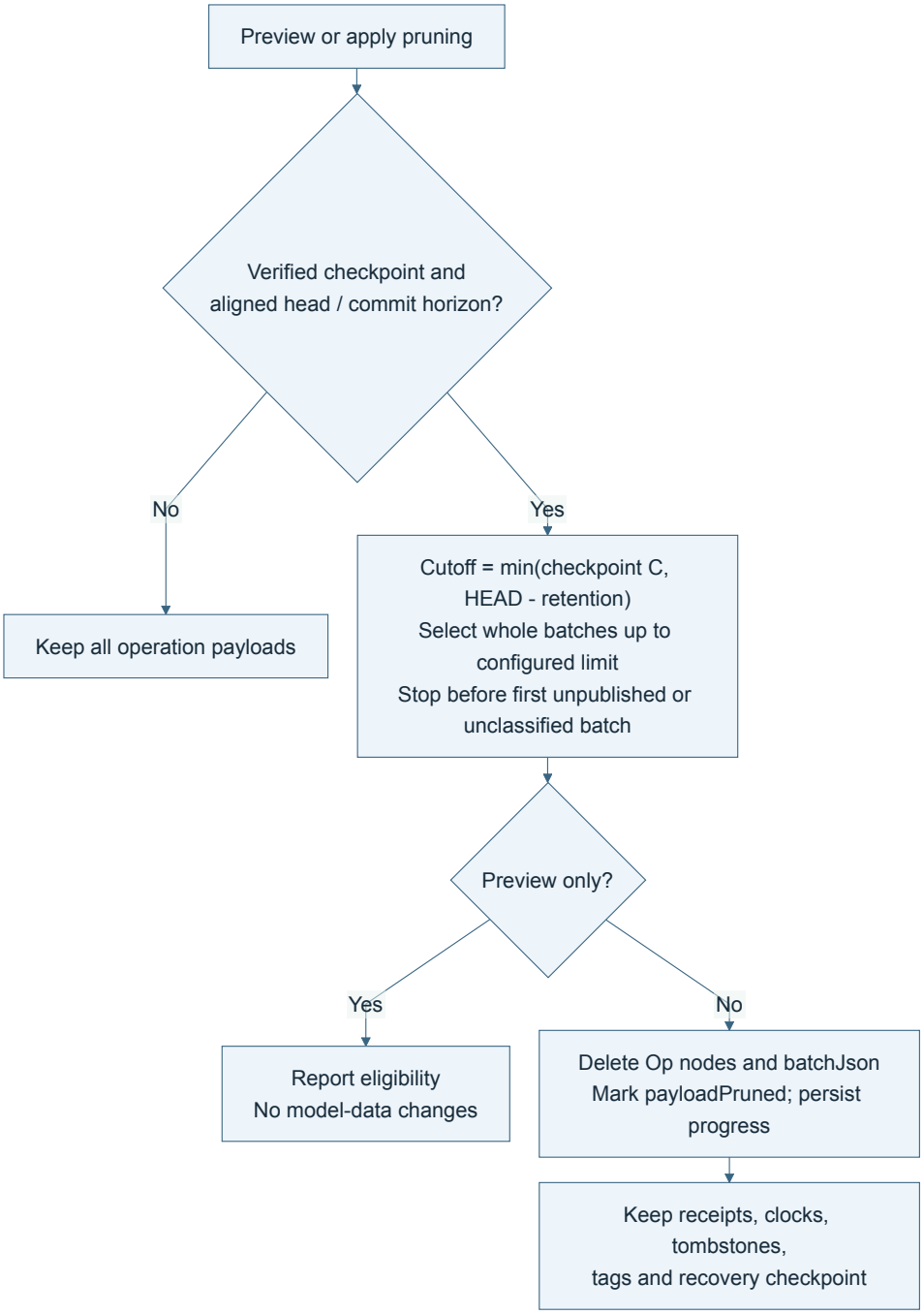


Figure 5: Payload pruning gates, publication fencing and retained safety state.

checkpoint, or since revision zero for the first one. Empty models are not automatically checkpointed. Models below the threshold can still finish pruning if they already have a checkpoint.

Payload retention uses `app.compaction.retain-revisions` (`ARCHIMESH_COMPACTION_RETAIN_REVISIONS`, default 1000). The per-model pruning limit is `app.compaction.max-batches-per-run` (`ARCHIMESH_COMPACTION_MAX_BATCHES`, default 100). Enabled scheduler policy values must be positive; revision retention may be zero.

```
export ARCHIMESH_CHECKPOINT_MAINTENANCE_ENABLED=true
export ARCHIMESH_CHECKPOINT_INTERVAL_SECONDS=300
export ARCHIMESH_CHECKPOINT_MIN_REVISIONS=1000
export ARCHIMESH_CHECKPOINT_RETAIN_COUNT=2
export ARCHIMESH_CHECKPOINT_MODELS_PER_RUN=10
export ARCHIMESH_COMPACTION_RETAIN_REVISIONS=1000
export ARCHIMESH_COMPACTION_MAX_BATCHES=100
```

Listing 40: Example opt-in policy, exported before server startup

Each visit takes the service's timeline serialization and checks the writer lease. It optionally creates a checkpoint, prunes one bounded payload set, then removes up to 100 older checkpoints while retaining the newest configured count. Cleanup verifies the latest checkpoint, contiguous acceptance receipts and complete tail first. The keep count must be at least one. Manual checkpoints participate in the same retention policy when scheduling is enabled.

These phases commit separately. A later failure leaves earlier safe work committed; the next visit resumes. Individual model failures are logged and do not starve other models. Lease loss stops processing further models, and the database mutations also check the lease. Significant results appear in server logs and model activity. The worker stops on shutdown. Without scheduling, older checkpoints accumulate; acceptance receipts and merge metadata always accumulate in the current implementation. See `docs/checkpoint-pruning.md` for the corresponding Markdown runbook.

11.5 Export

Export produces a self-contained package for one model timeline, suitable for backup or migration.

```
curl -s http://localhost:8081/admin/models/demo/export | jq .
```

Listing 41: Export a model

The export package contains:

`format` Package format identifier (`archi-model-export-v1` or `archi-model-export-v2`).
`exportedAt` UTC timestamp.
`model` Model catalog metadata (modelId, modelName, headRevision).
`accessControl` Per-model ACL snapshot (optional).
`snapshot` Materialized current state at HEAD. Contains folders, elements, relationships, views, view objects, connections, folder memberships and view object child members.
`checkpoint`, `receipts` V2 recovery state and prefix acceptance ranges.
`opBatches` Ordered complete history (v1) or checkpoint tail (v2). Each batch contains modelId, opBatchId, baseRevision, assignedRevisionRange, timestamp and the ops array with normalized causal metadata.
`tags` Immutable tags with their historical snapshots.

The package format is documented in detail in `server/EXPORT_FORMAT.md`.

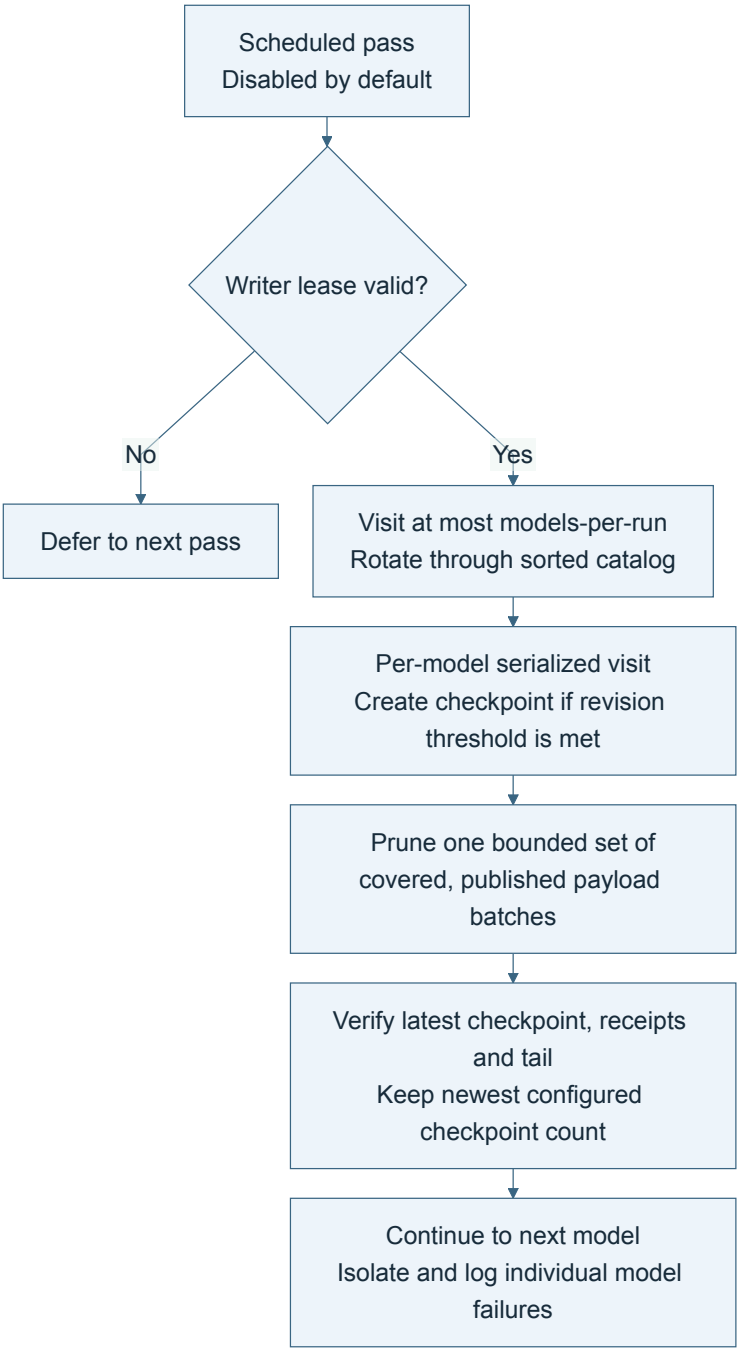


Figure 6: Optional per-model maintenance phases and policy gates.

11.5.1 What export preserves

- Model id and name
- Head revision
- Complete recovery data: full history or checkpoint plus tail and receipts
- Current materialized state
- Folders and folder memberships
- Views, view objects, connections and notation payloads
- Immutable tags and their historical snapshots
- Model-scoped ACLs

11.6 Import

Import restores a model from an export package.

```
curl -s -X POST http://localhost:8081/admin/models/import \
  -H "Content-Type: application/json" \
  -d @demo-export.json | jq .
```

Listing 42: Import a model

11.6.1 Import behavior

- The `format` field must be `archi-model-export-v1` or `archi-model-export-v2`.
- `model.modelId` must be present.
- Existing models are rejected unless `?overwrite=true`.
- Overwrite is rejected if the target model has active sessions.
- V2 overwrite restores the destination in one transaction; failure rolls it back.
- ACLs are restored if present in the package.
- V1 replays complete history; v2 restores the raw checkpoint, prefix receipts and complete tail. The resulting head matches the exported head.
- Tags are restored after the model timeline.

11.6.2 Overwrite with import

```
curl -s -X POST "http://localhost:8081/admin/models/import?overwrite=true" \
  -H "Content-Type: application/json" \
  -d @demo-export.json | jq .
```

Listing 43: Import with overwrite

11.6.3 Import requirements

V1 requires complete op-log history for non-empty models. V2 requires a verified checkpoint at C, acceptance receipts covering 1..C, and a complete tail C+1..HEAD. The tail can be empty when C equals HEAD. Tags retain their stored historical snapshots independently of pruned operations.

To restore v2 under another model ID, change only `model.modelId`. The importer verifies the original checkpoint, rebinds model-scoped state, tail batches and tag snapshots, and computes the destination checkpoint's checksum. Do not manually edit checkpoint state or its checksum. V2 packages preserve recovery state and timeline identity; they omit individual operation payloads at or before C.

11.7 Rebuild

Rebuild verifies the latest checkpoint and its complete operation tail before replacing the materialized projection in one transaction. Without a checkpoint, it requires the complete log from revision 1. A failed replacement rolls back.

```
curl -s -X POST http://localhost:8081/models/demo/rebuild | jq .
```

Listing 44: Rebuild a model

Or via the admin endpoint:

```
curl -s -X POST http://localhost:8081/admin/models/demo/rebuild-and-status | jq .
```

Listing 45: Rebuild and get status

Rebuild behavior:

1. Clears all materialized nodes for the model (elements, relationships, views, view objects, connections, folders and metadata).
2. Restores the raw checkpoint, or creates built-in roots when no checkpoint exists.
3. Replays the complete tail after the checkpoint in revision order.
4. Updates head revision to match the highest committed revision.
5. Tags are unaffected (they are stored separately).

11.8 Tags

Tags are immutable named pointers to historical revisions. They let you capture known-good states for later reference or checkout.

11.8.1 Creating a tag

```
curl -s -X POST http://localhost:8081/admin/models/demo/tags \
  --get --data-urlencode 'tagName=v1.2' \
  --data-urlencode 'description=Sprint 12 baseline' | jq .
```

Listing 46: Create a tag

The tag captures the current HEAD revision and stores a snapshot of the materialized state at that point.

11.8.2 Reading at a tag

Clients can join at a tagged revision by specifying the tag name as the `ref` in the `Join` message. The snapshot endpoint also accepts `?ref=<tagName>`:

```
curl -s "http://localhost:8081/models/demo/snapshot?ref=v1.2" | jq .
```

Listing 47: Read snapshot at tag

Tagged refs are read-only. Writes, locks and presence are only allowed on `HEAD`.

11.8.3 Tag naming

Use descriptive names that remain meaningful outside the codebase:

- `v1.2` — version numbers
- `milestone-3` — project milestones
- `approved-2026-03-08` — date-stamped approvals

Tag deletion is disabled by default. Re-enable it only with `app.tags.allow-delete=true` if you explicitly want mutable tag administration.

11.9 Integrity checks

The integrity endpoint checks for structural problems in the materialized graph:

```
curl -s http://localhost:8081/admin/models/demo/integrity | jq .
```

Listing 48: Check integrity

The response reports:

- Missing references: entities that reference ids not present in the graph (e.g. a view object whose `representsId` points to a deleted element).
- Orphan nodes: materialized nodes not reachable from the model root.
- Dangling relationships: relationships where source or target element is missing.

Integrity issues indicate either a bug in the apply path or manual Neo4j modifications. If issues are found, run a rebuild to reconstruct the materialized state from a verified checkpoint plus complete tail, or full log when no checkpoint exists. Invalid recovery coverage is rejected before projection replacement.

11.10 Consistency checks

Optional post-commit consistency checks can be enabled with `ARCHIMESH_CONSISTENCY_CHECK=true`. When enabled, the server runs a lightweight consistency verification after each `SubmitOps` commit, checking head alignment and referential integrity. This is not a full state comparison against a fresh replay. Failures are logged at WARN level.

This is disabled by default because it adds latency to every submit. Enable it during debugging or after infrastructure changes.

11.11 ACL management

Per-model ACLs restrict model access to specific users:

```
curl -s http://localhost:8081/admin/models/demo/acl | jq .
```

Listing 49: Read ACL

```
curl -s -X PUT http://localhost:8081/admin/models/demo/acl \
  -H "Content-Type: application/json" \
  -d '{"adminUsers":["alice"],"writerUsers":["alice","bob"],"readerUsers":["carol"]}'
    | jq .
```

Listing 50: Update ACL

When a model has a configured ACL, it overrides generic reader/writer role checks for that model. Catalog-wide admin actions still require the global admin role.

11.12 Status monitoring

Check model health at a glance:

```
curl -s http://localhost:8081/admin/models/demo/status | jq .
```

Listing 51: Model status

The response includes:

- `modelId`, `modelName`
- `headRevision`
- Element, relationship, view, folder, view object and connection counts
- Consistency check results
- Use the model window and checkpoint summary for maintenance activity; pruning responses carry the actual watermark and payload deletion counts.
- Use the sessions endpoint for active session details.

12 Troubleshooting

This chapter covers common failure modes, diagnostic approaches and recovery procedures.

12.1 Diagnostic endpoints

Several endpoints help diagnose problems without restarting the server:

Endpoint	Purpose
`GET /admin/overview`	Check which models exist and their health. Look for stale head revisions or integrity warnings.
`GET /admin/models/\{modelId\}/status`	Detailed model status: counts, revision, consistency.
`GET /admin/models/\{modelId\}/integrity`	Structural integrity: missing references, orphan nodes.
`GET /admin/auth/diagnostics`	Authorization troubleshooting: resolved identity, roles, mode.
`GET /admin/sessions`	Active WebSocket sessions across all models. Check for stale or abandoned sessions.
`GET /admin/audit/config`	Current audit configuration.

12.2 Common problems

12.2.1 Client cannot connect

Symptom Connection fails with "Failed to connect archimesh websocket".

Possible causes

- Server is not running. Check with `curl http://localhost:8081/admin/overview`.
- Wrong URL. The WebSocket URL should be `ws://<host>:<port>` (not `http`).
- Authorization failure. Check the auth token validity and role assignments. Use `GET /admin/auth/diagnostics`.
- Model does not exist. Models must be registered via `POST /admin/models/\{modelId\}` before clients can connect.
- Network firewall blocking WebSocket upgrade.

12.2.2 SubmitOps rejected

Symptom The server replies with a correlated `Error` to the client. The status line shows a conflict banner.

Error codes **`REVISION_AHEAD`** Client's `baseRevision` exceeds server head. The client is ahead of the server — this usually means the client missed a reconnect after a server state change. Solution: the client should resynchronize with the server. The rejection is sent to the submitting session and identifies its batch where available.

`PRECONDITION_FAILED` A referenced entity does not exist. The op references an element, relationship, view, view object or connection that is not in the materialized graph and was not created earlier in the same batch. Common causes: deleting an entity on one client while another client edits a diagram object

representing it; submitting a connection create before the relationship is committed. The conflicting outbox entry is dropped and surfaced in the UI.

`LOCK_CONFLICT` Another client holds a notation lock on one of the targets. The client should retry after releasing and re-acquiring the lock. The conflicting outbox entry is dropped.

12.2.3 Materialized state corrupted

Symptom Integrity check reports missing references or orphan nodes.

Cause Bug in the apply path, manual Neo4j modification or failed partial writes.

Solution Run a rebuild:

```
curl -s -X POST http://localhost:8081/models/demo/rebuild
```

Verification After rebuild, run integrity check again and verify `issues` is empty.

12.2.4 Neo4j connection failure

Symptom Server logs show Neo4j connection errors. All operations fail.

Check Is Neo4j running? `docker compose ps neo4j`. Check the Neo4j browser at http://localhost:7474.

Check Are credentials correct? Verify `NEO4J_URI`, `NEO4J_USER` and `NEO4J_PASSWORD` match the running instance.

Check Is the Bolt port accessible? Default is 7687.

12.2.5 Kafka unavailable

Symptom Server logs show Kafka publish warnings. Other clients do not receive real-time updates from the submitter's client.

Impact After a successful Neo4j commit, submitters can receive `OpsAccepted` while other clients await fan-out. Pending payloads remain durable and are retried in order after broker recovery or server restart. A retry may duplicate a broadcast. Pruning cannot remove pending publication payloads.

Check Is Kafka running? `docker compose ps kafka`. Check the Kafka UI at http://localhost:8080.

Check Do the required topics exist? Use

```
docker compose exec kafka rpk topic list --brokers localhost:9092.
```

12.2.6 Outbox filling up

Symptom The Archi status line shows "Outbox near capacity". Local edits are being queued because the client is disconnected or the connection is flapping.

Impact At 1000 entries, new enqueue is rejected and existing uncommitted work is preserved. The warning fires at 800 entries (80%).

Solution Reconnect to the server so the outbox can drain. If connectivity is persistently poor, work in smaller edit batches and reconnect frequently.

Prevention Monitor the server's session page for clients that repeatedly connect and disconnect. Investigate network stability.

12.2.7 Cannot overwrite a model

Symptom Import with `?overwrite=true` returns 409.
Cause Active sessions exist on the target model.
Solution Disconnect all clients from the model, verify with `GET /admin/sessions`, then retry. If a session is stuck, force delete the model and re-import.

12.3 Log analysis

The server emits structured logs. Key log categories:

Logger	Content
`org.gautelis.archimesh`	Application-level events: submit, accept, reject, join, disconnect, rebuild, compact, export, import, tags.
`org.apache.kafka`	Kafka client logs (WARN by default).
`admin_audit` (INFO)	Admin API access audit events.
`ws_audit` (INFO)	WebSocket lifecycle audit events.

Set `ARCHIMESH_LOG_LEVEL=DEBUG` for detailed flow tracing including op-level detail. Note that DEBUG logging of `OpsBroadcast` messages can produce very large log volumes on active models.

Log format:

```
1 yyyy-MM-dd HH:mm:ss.SSS LEVEL [logger.] (thread) message
```

Listing 52: Log format

Audit events include a stable JSON payload with user id, action and request context. Parse the JSON rather than depending on free-form message text.

12.3.1 Client-side log analysis

The client plugin writes to the Archi workspace log (`~/Archi/workspace/.metadata/.log`). Search for:

- `archimesh` — all plugin log entries
- `Connected archimesh` — successful connections
- `SubmitOps received` — server acknowledging submits
- `Queued SubmitOps` — entries moved to outbox
- `Outbox full` — capacity reached; existing entries preserved
- `Dropping stale` — conflict resolution dropping an entry
- `Error stopping Archimesh` — shutdown errors

The `scripts/check-archimesh-sanity.sh` script scans Archi workspace logs for critical sync issues, integrity warnings and connection errors.

12.4 Recovery procedures

12.4.1 Repeatable process-crash drill

Run `./scripts/validate-local.sh recovery` to exercise packaged server JVMs against isolated Docker infrastructure. The drill force-stops an accepted but unpublished batch, recovers delivery after Kafka returns, restarts after checkpoint pruning and tail acceptance, and restores the package into an empty database. A database uniqueness lock synchronizes a forced stop inside the restore transaction; no partial model may remain after rollback.

The drill compares snapshots, tags, checkpoint hashes and old acceptance ranges, then checks stale-write/delete barriers, further edits and rebuild. It waits for the real writer lease instead of resetting it. Reports, fixture exports and process logs are saved under `server/target/recovery-drill/<run-uuid>/`. Ordinary gates skip this opt-in test. Default cleanup removes only the drill's own containers, volumes and child processes; the existing development stack is unaffected. See `docs/recovery-drills.md` for commands, retained-infrastructure diagnostics and the limits of the tested failure points.

12.4.2 Pruning makes no progress

Inspect the compact response's `note`, `checkpointRevision`, `watermarkRevision` and `eligibleBatchCount`. With no checkpoint, all payloads remain. A recent-revision retention window can legitimately leave no whole eligible batch. The first unpublished or unclassified legacy batch blocks its own payload and every later one; restore Kafka delivery before retrying. Head/commit mismatch disables pruning.

12.4.3 Checkpoint or recovery coverage rejected

A damaged checksum, unsupported raw state, missing receipt interval or incomplete recovery tail is rejected. Do not bypass validation by clearing merge metadata or changing publication flags. Restore a known-good export or database backup. Data removed by an older server before any recovery checkpoint cannot be reconstructed from a partial log. Rebuild validates recovery data before replacement and rolls back transaction failures.

12.4.4 Scheduled maintenance does not run

It is disabled by default. Verify `ARCHIMESH_CHECKPOINT_MAINTENANCE_ENABLED`, the interval and revision threshold, and the server's writer lease. The first pass waits one interval; large catalogs rotate across passes. Search for `Checkpoint maintenance` logs. Model-specific failures are isolated; later phases can be deferred even if earlier safe phases committed. See Section 11.4.

12.4.5 Server restart

If the server is restarted:

1. All WebSocket connections are closed. Clients will show "Disconnected" in the status line.
2. Client outboxes start queuing local edits.
3. When the server is back up, clients must manually reconnect.
4. On reconnect, the client sends a `Join` with its cached revision. The server sends a delta (if the revision gap is within the op-log window) or a full snapshot.

5. The outbox drains automatically after reconnect and revision hints are available.

12.4.6 Neo4j data loss

If the Neo4j database is lost or irrecoverably corrupted:

1. Restore from the most recent export package using `POST /admin/models/import`.
2. If no database backup or export exists, server checkpoints are lost with that database. Preserve any local Archi files before reconnecting; they cannot reconstruct the old accepted timeline or receipts.
3. After restoring the server model, clients reconnect and receive a complete delta or snapshot. A new empty model does not recover the old timeline automatically from a client cache.

12.4.7 Full system recovery

In a disaster scenario where both Neo4j and the server are lost:

1. Restore infrastructure: `docker compose up -d`.
2. Apply Neo4j schema: `docker compose exec -T neo4j cypher-shell ... < neo4j/schema.cypher`.
3. Start the server.
4. Import the most recent model exports.
5. Clients reconnect and receive snapshots from the restored server.

12.5 Performance tuning

- **Neo4j heap:** The materialized graph and op-log grow with model activity. Monitor Neo4j heap usage and increase `NEO4J_dbms_memory_heap_max__size` as needed.
- **Kafka retention:** Long Kafka retention on ops topics consumes disk. If rebuild is done from Neo4j (not Kafka), set shorter retention for ops topics.
- **Compaction frequency:** Run compaction periodically on active models after creating a verified checkpoint to reclaim published payloads. Review eligible and pruned batch counts; receipts and merge metadata remain.
- **Poll interval:** The admin UI polls the server for status updates. Set the poll interval to 0 to disable auto-refresh if dashboard traffic is a concern.
- **Consistency checks:** Keep `ARCHIMESH_CONSISTENCY_CHECK` disabled in production unless debugging. It adds latency to every submit.

13 Script Reference

This appendix documents all scripts in the `scripts/` directory.

The manual has its own reproducible build helpers: `bash docs/manual/build.sh` builds from checked-in vector figures; `bash docs/manual/build.sh --render-figures` regenerates the Mermaid figures before compiling. Sources and prerequisites are documented in `docs/manual/README.md`.

13.1 Build and install

`build-archimesh-client.sh` Convenience wrapper that calls `build-client-standalone.sh`.

`build-client-standalone.sh` Builds the client plugin using Tycho/Maven with the target platform.

`build-admin-web.sh` Builds the Svelte admin UI and copies output to the server's static resources directory.

`install-archimesh-plugin.sh` Installs the built client plugin JAR into the local Archi installation's dropins directory.

`sync-archimesh-client.sh` Syncs client plugin source changes without a full rebuild.

`prepare-client-target-platform.sh` Prepares the Eclipse target platform bundles required for client plugin compilation.

`mint-dev-jwt.sh` Mints a local dev JWT signed with the test private key.
Usage: `./mint-dev-jwt.sh --user alice --roles admin,model_writer`
Options: `--expires-in <seconds>`, `--audience <aud>`, `--print-payload`.
Env overrides: `DEV_JWT_PRIVATE_KEY`, `DEV_JWT_ISSUER`, `DEV_JWT_EXPIRES_IN`, `DEV_JWT_AUDIENCE`.

13.2 Development infrastructure

`dev-up.sh` Starts Docker Compose infrastructure (Neo4j + Kafka) and optionally the server. Usage: `./dev-up.sh [background]`. Supports OIDC mode via `ARCHIMESH_IDENTITY_MODE=oidc`.

`dev-down.sh` Stops Docker Compose infrastructure.

13.3 Validation and gate checks

`run-isolated-server-gate.sh` Runs all server unit, infrastructure and WebSocket tests on fresh disposable Neo4j/Kafka with an ephemeral HTTP test port. It requires Python 3 for run IDs and port selection, removes only its own Docker project, and saves logs and Surefire reports under `server/target/isolated-server-gate/`. Equivalent entry point: `./scripts/validate-local.sh server-isolated`.

`run-recovery-drill.sh` Packages the server and runs the opt-in `ProcessRecoveryIT` against dedicated Docker infrastructure. It force-stops only its own child server JVMs, waits for real writer-lease expiry, checks pending publication and checkpoint recovery, and interrupts an atomic v2 restore into an empty database. Evidence is retained under `server/target/recovery-drill/`. Use `--skip-build` to reuse an existing package. The equivalent gate entry point is `./scripts/validate-local.sh recovery`; see `docs/recovery-drills.md` for scenarios and cleanup.

validate-local.sh Central validation entry point.
Usage: `./validate-local.sh <mode>`
Modes:

fast Lightweight hardening and parity checks (no Docker infra needed). Runs `check-crdt-hardening.sh` and `check-notation-parity.sh`.

gate Full client + server CRDT gate via `crdt-gate.sh`.

infra Full gate with local Kafka/Neo4j-backed integration coverage. Sets `RUN_LOCAL_INFRA_IT=true`.

ws Full gate with websocket end-to-end coverage. Sets `RUN_WS_E2E_IT=true`.

recovery Packaged JVM crash and fresh-database restore drill against dedicated Docker infrastructure.

server-isolated All server tests on fresh disposable Neo4j/Kafka, including WebSocket coverage; client tests run separately.

crdt-gate.sh Runs the CRDT test suite end-to-end. First runs hardening contract checks, then installs the client plugin artifact, runs client CRDT tests (`OpMapperNotationInventoryTest`, `OpMapperConnectionBatchTest`, `CrdtEntityMergeTest`, `CrdtPropertyMergeTest`, `CrdtOrSetTest`), then runs server CRDT tests (`ArchimeshServiceTest`, optionally `LocalInfraIntegrationTest` when `RUN_LOCAL_INFRA_IT=true`).

check-crdt-hardening.sh Preflight contract check that verifies presence of core CRDT test files, gate scripts and spec references. Fails fast when test infrastructure drifts.

check-notation-parity.sh Compares notation key inventory across: schema definitions (`schemas/ops.json`), server runtime whitelist constants (`ArchimeshService`) and client inventory test expectations (`OpMapperNotationInventoryTest`). Ensures all three agree on supported notation fields.

check-archimesh-sanity.sh Scans Archi workspace logs for critical sync issues, integrity warnings and connection errors. Useful for diagnosing client-side problems.

13.4 Log monitoring

tail-archimesh-logs.sh Tails Archi workspace logs with archimesh-related filtering. Useful for real-time debugging during client plugin development.

14 Wire Protocol and Schema Reference

This appendix documents the JSON schemas that define the Archimesh wire protocol contract. The authoritative schema files live in `schemas/`.

14.1 Schema files

File	Purpose
`common.json`	Shared type definitions: `ModelId`, `UserId`, `SessionId`, `EntityId`, `RFC3339`, `Causal`, `Actor`, `RevisionRange`, `JsonObject`.
`ops.json`	Operation type definitions for all supported op types: create, update, delete for elements, relationships, views, view objects, connections, folders; property and membership operations; plus strict notation JSON objects for view objects and connections.
`op-batch.json`	Op batch envelope: `SubmitOps` with `baseRevision`, `opBatchId`, `actor` and `ops` array.
`lock-event.json`	Lock event payload: action (acquired, released, expired), actor, targets, timestamp.
`presence.json`	Presence payload: modelId, userId, sessionId, viewId, selection array, cursor object.
`notation-contract.md`	Human-readable contract describing the notation field boundaries and parity requirements.

14.2 Message flow summary

Direction	Type	Description
Client → Server	`Join`	Request checkout. Optional `lastSeenRevision` and `ref`.
Client → Server	`SubmitOps`	Submit one op batch. `baseRevision`, `opBatchId`, `ops`.
Client → Server	`AcquireLock`	Acquire notation lock on targets. `ttlMs`.
Client → Server	`ReleaseLock`	Release notation lock on targets.
Client → Server	`Presence`	Broadcast cursor position and selection on a view.
Server → Client	`CheckoutSnapshot`	Full materialized model state.
Server → Client	`CheckoutDelta`	Incremental ops since `lastSeenRevision`.
Server → Client	`OpsAccepted`	Batch commit confirmation with assigned revision range.
Server → Client	`OpsBroadcast`	Remote batch broadcast (Kafka fan-out through the consumer).
Server → Client	`LockEvent`	Lock state change notification.
Server → Client	`PresenceBroadcast`	Remote cursor/selection update.
Server → Client	`Error`	Rejection or failure with `code` and `message`.

14.3 Common type definitions

14.3.1 Causal metadata

```
{
  "clientId": "<stable client or session identifier>",
  "lamport": <monotonic integer per client>,
  "opId": "<globally unique operation identity>"
}
```

Listing 53: Causal object

14.3.2 Actor

```
{
  "userId": "<user identifier>",
  "sessionId": "<session identifier>"
}
```

Listing 54: Actor object

14.3.3 Revision range

```
{
  "from": <inclusive start revision>,
  "to": <inclusive end revision>
}
```

Listing 55: Revision range object

14.4 Op batch envelope

```
{
  "type": "SubmitOps",
  "payload": {
    "baseRevision": <client revision>,
    "opBatchId": "<UUID>",
    "actor": { "userId": "...", "sessionId": "..." },
    "ops": [
      { "type": "CreateElement", ... },
      { "type": "UpdateViewObjectOpaque", ... }
    ]
  }
}
```

Listing 56: SubmitOps payload

14.5 Notation JSON objects

View object and connection notation are validated against strict schemas (`additionalProperties: false`):

14.5.1 ViewObjectNotationJson

Supported fields: `x`, `y`, `width`, `height`, `type`, `alpha`, `lineAlpha`, `lineWidth`, `lineStyle`, `textAlignment`, `textPosition`, `gradient`, `iconVisibleState`, `deriveElementLineColor`, `fillColor`, `lineColor`, `font`, `fontColor`, `iconColor`, `imagePath`, `imagePosition`, `name`, `documentation`.

14.5.2 ConnectionNotationJson

Supported fields: `type`, `nameVisible`, `textAlignment`, `textPosition`, `lineWidth`, `name`, `lineColor`, `font`, `fontColor`, `documentation`, `bendpoints`.

14.6 Error codes

Code	Meaning
`REVISION_AHEAD`	Client `baseRevision` exceeds server head. Client must rejoin.
`PRECONDITION_FAILED`	Referenced entity does not exist, or notation contains unknown keys.
`LOCK_CONFLICT`	Another client holds a conflicting notation lock.
`MODEL_NOT_FOUND`	The requested model is not registered.
`REF_NOT_FOUND`	The requested tag name does not exist.
`REF_READ_ONLY`	Write attempted on a tagged (read-only) ref.

14.7 Export/import format

Without a checkpoint the server emits a v1 package containing:

```
{
  "format": "archi-model-export-v1",
  "exportedAt": "<UTC timestamp>",
  "model": { "modelId": "...", "modelName": "...", "headRevision": 42 },
  "accessControl": { "configured": true, "adminUsers": [...], ... },
  "snapshot": { "format": "archimate-materialized-v1", ... },
  "opBatches": [ { "modelId": "...", "opBatchId": "...", ... } ],
  "tags": [ { "tagName": "...", "revision": 24, "snapshot": {...} } ]
}
```

Listing 57: Export package structure

The package format is documented in full in `server/EXPORT_FORMAT.md`.

Checkpointed exports use `archi-model-export-v2`. They add `checkpoint` (format, model ID, revision C, timestamp, checksum and raw state) and `receipts` (batch IDs, original ranges and timestamps covering 1..C). Their `opBatches` contain exactly the tail C+1..HEAD, which can be empty at C=HEAD. V1 remains supported. See Section 11 for atomic v2 restore, target model rebinding and full-history archive requirements.